udp UNIVERSIDAD DIEGO PORTALES

RECONOCIMIENTO DE PATRONES EN IMÁGENES

FACULTAD DE INGENIERÍA Y CIENCIAS
UNIVERSIDAD DIEGO PORTALES

2DO SEMESTRE 2025

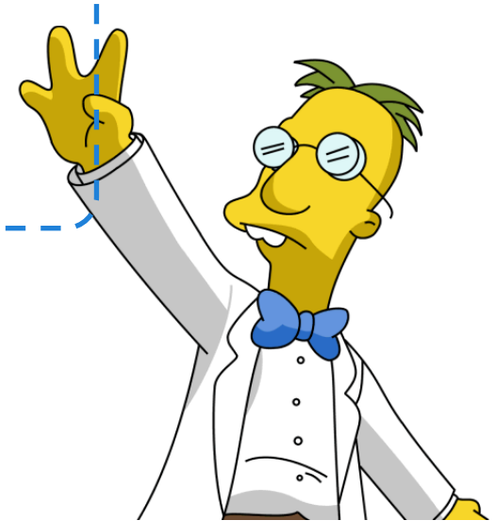
PROFESOR: MIGUEL CARRASCO

CARACTERÍSTICAS GEOMÉTRICAS

Background https://i0.wp.com/ubiai.tools/wp-content/uploads/2023/11/pattern-recognition-image.jpg?fit=2234%2C1180&ssl=1

Extracción decaracterísticas | Panorama general

- Geométricas
  - Centro de masa
  - Tamaño
  - Perímetro
  - Redondez
  - Momentos binarios
  - Descritores de Fourier
  - Elipse
  - Distancia al borde
- Cromáticas
  - Color promedio
  - Gradiente promedio
  - Promedio segunda derivada
  - Contraste
  - Momentos de color
  - Textura
- Otros descriptores
  - PHOG
  - SURF
  - SIFT
  - GPD



- Características geométricas

Centro de Masa

Tamaño

Perímetro

Redondez

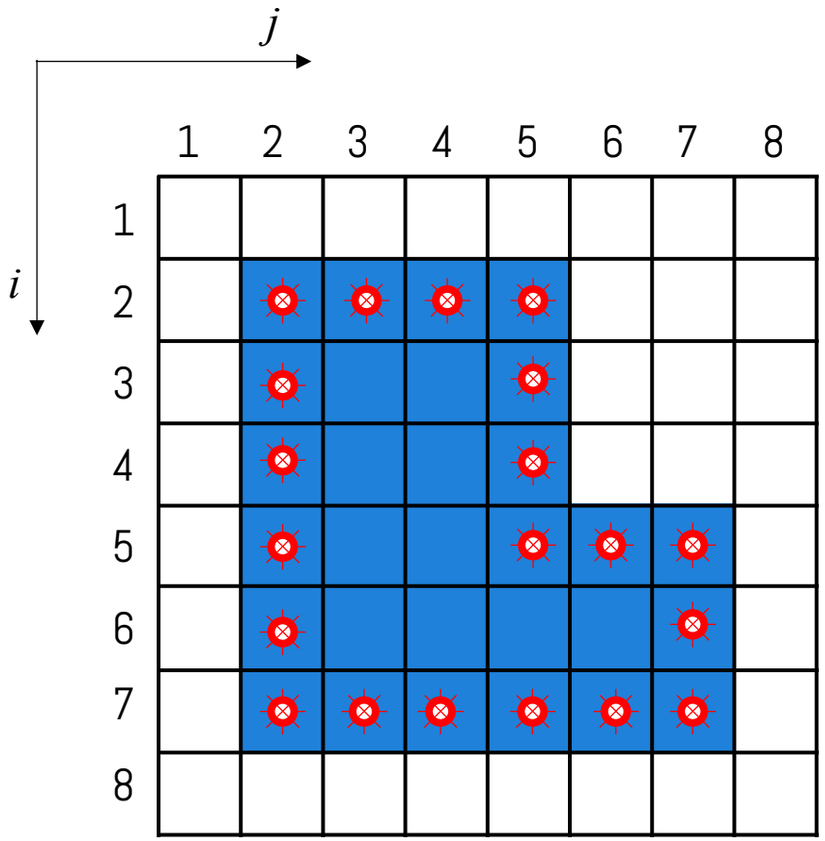
Momentos

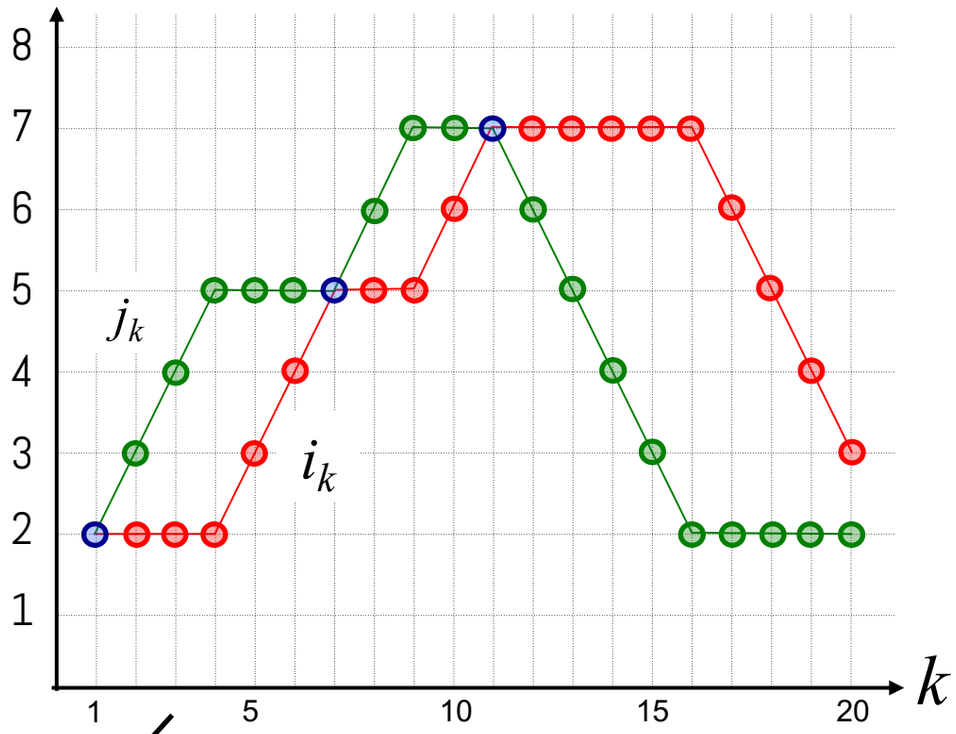
Descriptores de Fourier

Elipse

Distancias

- Los descriptores de Fourier corresponden al módulo de los coeficientes complejos de Fourier de las posiciones de borde de la región.





Los píxeles de los bordes deben ser ordenados en un sentido de giro

- Características geométricas

Centro de Masa

Tamaño

Perímetro

Redondez

Momentos

Descriptores de Fourier

Elipse

Distancias

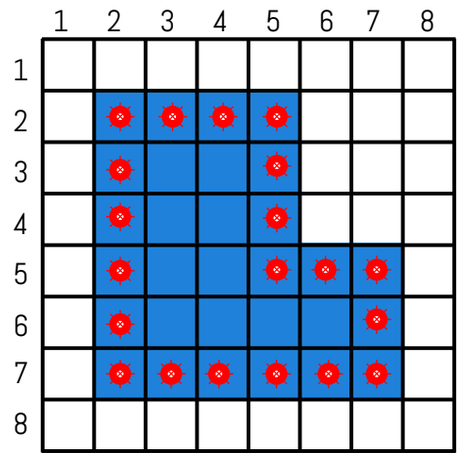
- La función `findContours` permite obtener las coordenadas de la frontera de una región binaria. Fijarse en los parámetros para obtener las coordenadas

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

bw = [ [0, 0, 0, 0, 0, 0, 0, 0],
       [0, 1, 1, 1, 1, 0, 0, 0],
       [0, 1, 1, 1, 1, 0, 0, 0],
       [0, 1, 1, 1, 1, 0, 0, 0],
       [0, 1, 1, 1, 1, 1, 1, 0],
       [0, 1, 1, 1, 1, 1, 1, 0],
       [0, 1, 1, 1, 1, 1, 1, 0],
       [0, 0, 0, 0, 0, 0, 0, 0]]

binary = np.array(bw, dtype='uint8')

cnts, hc = cv2.findContours(binary,
                            cv2.RETR_EXTERNAL,
                            cv2.CHAIN_APPROX_NONE)
```



La función `findContours` determina los bordes de todas las regiones presentes en la imagen binaria

- Características geométricas

Centro de Masa

Tamaño

Perímetro

Redondez

Momentos

Descriptores de Fourier

Elipse

Distancias

- Una vez obtenidos las coordenadas, construimos un nuevo vector con los valores definidos en formato complejo. Note que hacemos la lista en forma circular.

```
cnts, hc = cv2.findContours(binary,
                            cv2.RETR_EXTERNAL,
                            cv2.CHAIN_APPROX_NONE)

contornos = np.vstack(cnts[0])

x , y = zip(*contornos)
CX = np.array(x) + 1j*np.array(y)
CX = np.append(CX, CX[0])
```

El vector de coordenadas lo transformamos en un vector de números complejos en forma circular (insertamos el primer elemento de la lista al final

coordenadas en forma compleja

$$CX = \begin{matrix} 1 + 1i \\ 1 + 2i \\ 1 + 3i \\ 1 + 4i \\ 1 + 6i \\ \\ \\ \\ \\ 3 + 2i \\ 2 + 2i \end{matrix}$$

- Características geométricas

Centro de Masa

Tamaño

Perímetro

Redondez

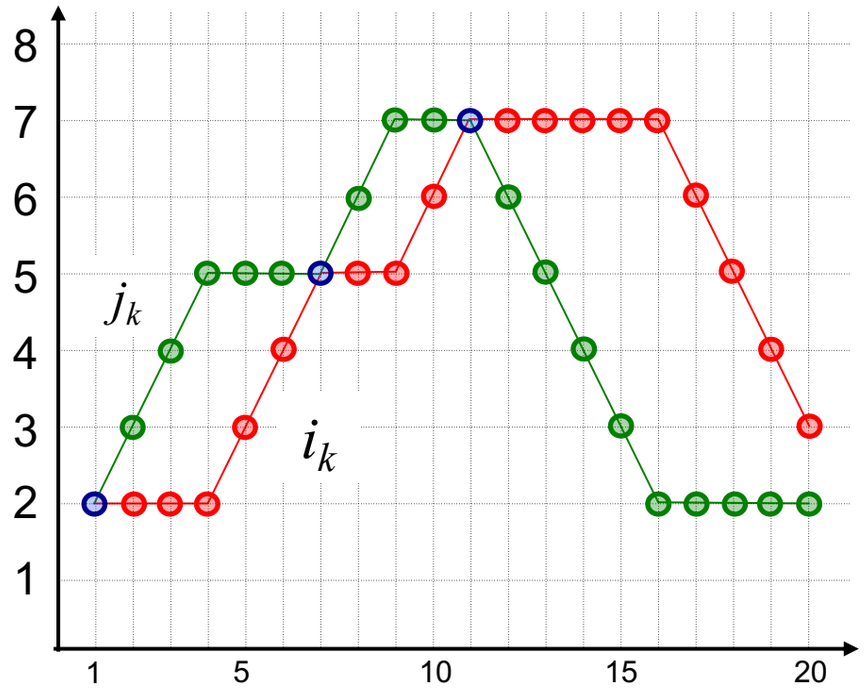
Momentos

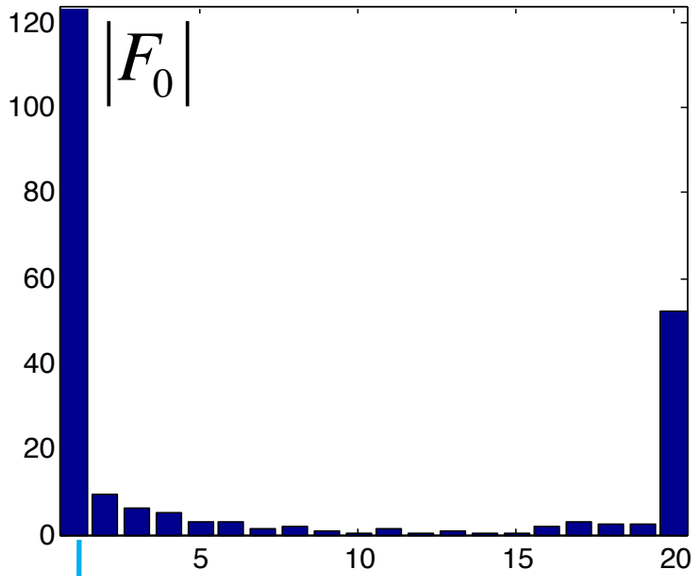
Descriptores de Fourier

Elipse

Distancias

- Posteriormente calculamos los descriptores de Fourier de la región, y determinamos los coeficientes de Fourier.





El primer descriptor da información sobre la posición de la región. Los siguientes descriptores son **invariantes** a la posición

- Características geométricas

Centro de Masa

Tamaño

Perímetro

Redondez

Momentos

Descriptores de Fourier

Elipse

Distancias

- Al transformar las posiciones espaciales de los bordes en una señal periódica, podemos emplear técnicas de filtraje en la frecuencia con la Transformada Discreta de Fourier

```
binary = np.array(bw, dtype='uint8')

cnts, _= cv2.findContours(binary,
                  cv2.RETR_EXTERNAL,
                  cv2.CHAIN_APPROX_NONE)


contornos = np.vstack(cnts[0])

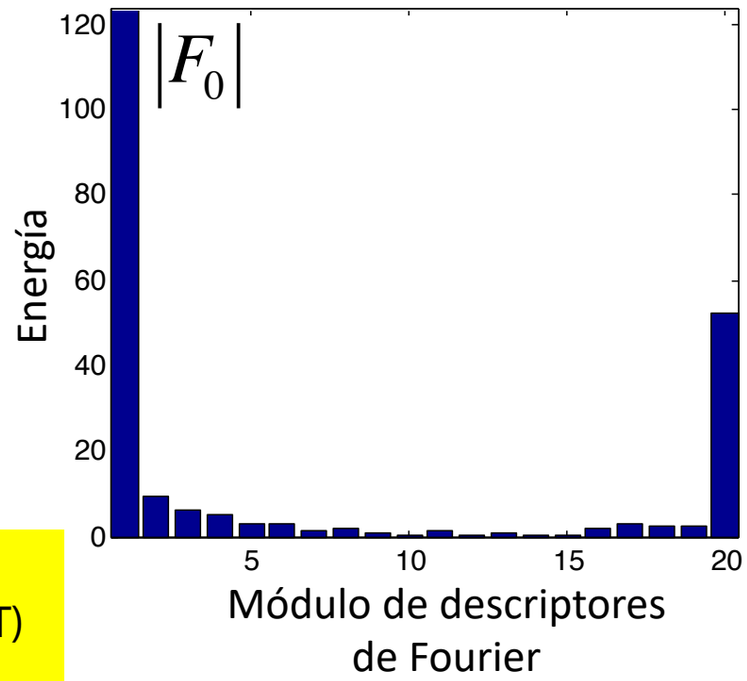
x , y = zip(*contornos)
CX = np.array(x) + 1j*np.array(y)
CX = np.append(CX, CX[0])

FF = np.fft.fft(CX)

plt.figure()
plt.bar(np.arange(len(FF)),np.abs(FF))
plt.show()
```

Fast Fourier Transform (FFT)

Gráfico de barras



- Características geométricas

Centro de Masa

Tamaño

Perímetro

Redondez

Momentos

Descriptores de Fourier

Elipse

Distancias

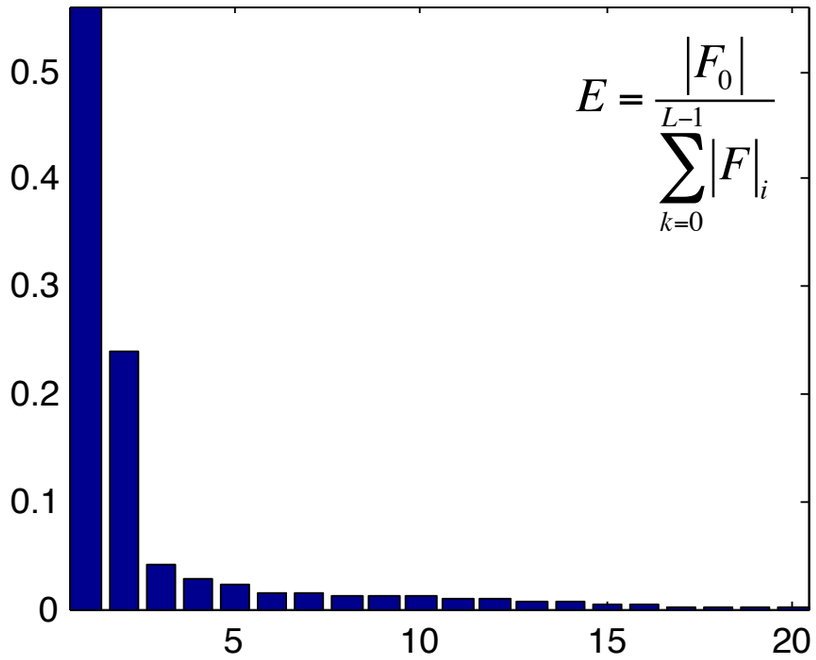
- Una vez calculados los descriptores, debemos seleccionar aquellos que provean más información (en caso que queramos modificar la forma de la región).

```
FF = np.fft.fft(CX)

#Energia
E = np.abs(FF)/np.sum(np.abs(FF))

#ordenamos los descriptores
idx = np.argsort(E)
idx = idx[::-1]

#graficamos
plt.figure()
plt.bar(np.arange(len(E)), E[idx])
plt.show()
```



Descriptores de Fourier ordenados de mayor a menor normalizados

- Características geométricas

Centro de Masa

Tamaño

Perímetro

Redondez

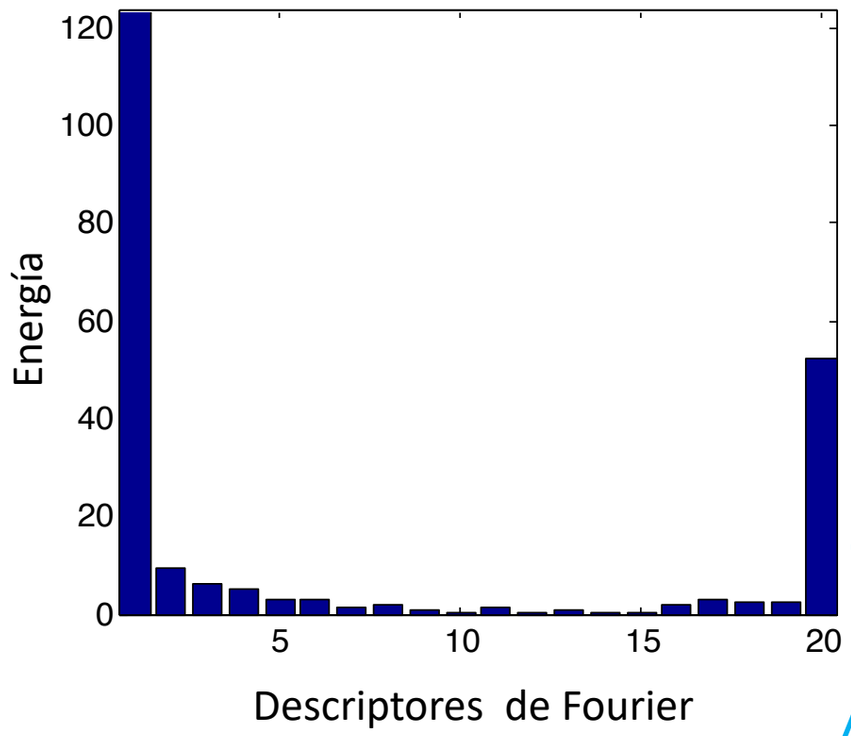
Momentos

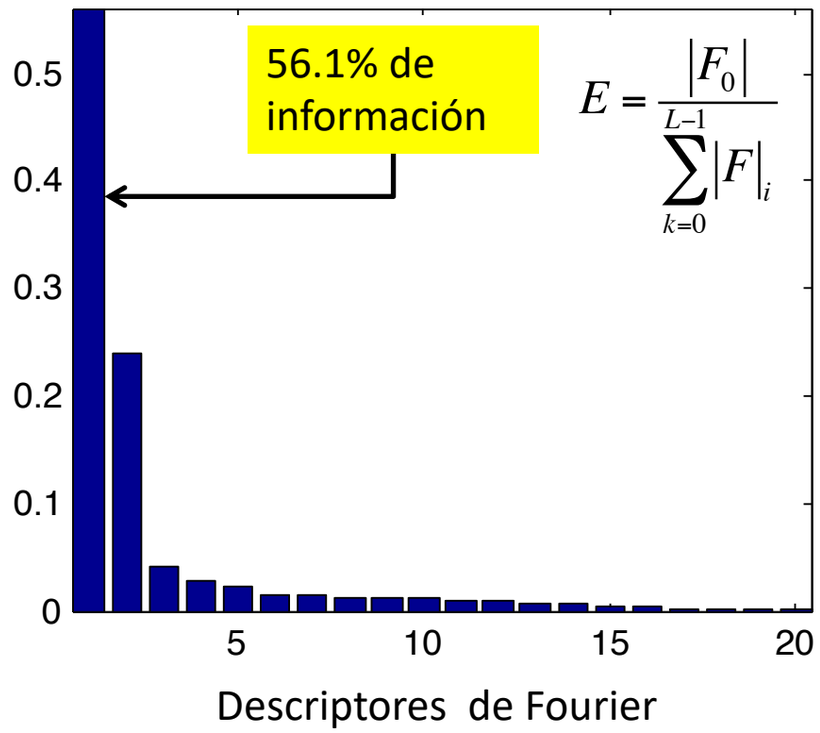
Descriptores de Fourier

Elipse

Distancias

- Una vez calculados los descriptores, debemos seleccionar aquellos que provean más información (en caso que queramos modificar la forma de la región).





Si la figura fuera un círculo perfecto, los descriptores de Fourier serían todos cero ya que corrresponderían a una sinusoide

- Características geométricas

Centro de Masa

Tamaño

Perímetro

Redondez

Momentos

Descriptores de Fourier

Elipse

Distancias

- Una vez calculados los descriptores, debemos seleccionar aquellos que provean más información (en caso que queramos modificar la forma de la región).

```
FF = np.fft.fft(CX)

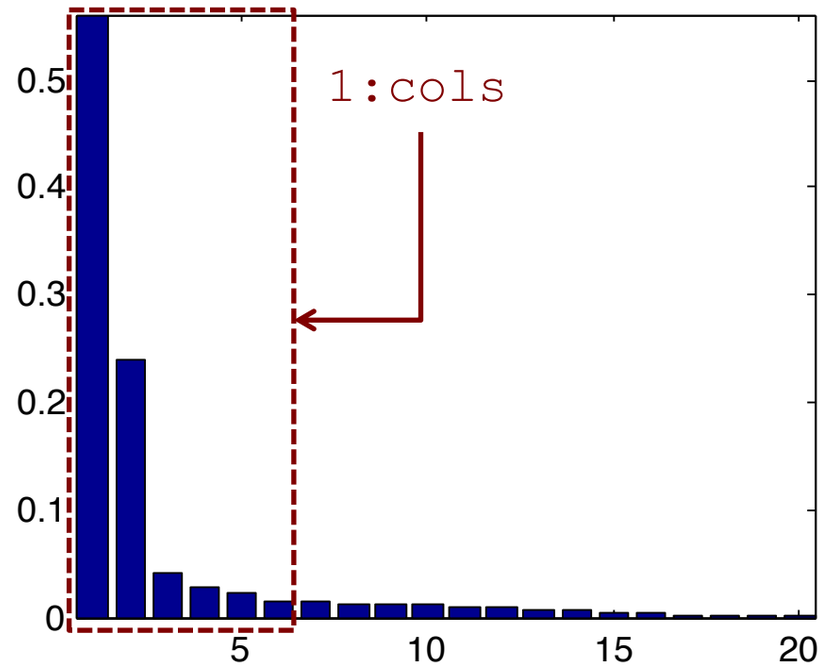
#Energia
E = np.abs(FF)/np.sum(np.abs(FF))

#ordenamos los descriptores
idx = np.argsort(E)
idx = idx[::-1]

#graficamos
plt.figure()
plt.bar(np.arange(len(E)), E[idx])
plt.show()

cols = 5
sel = idx[0:cols]
```

Seleccionamos los primeros descriptores (desde 0 hasta cols)



Descriptores de Fourier ordenados de mayor a menor normalizados

- Características geométricas



- Una vez calculados los descriptores, debemos seleccionar aquellos que provean más información (en caso que queramos modificar la forma de la región).

```
FF = np.fft.fft(CX)

#Energia
E = np.abs(FF)/np.sum(np.abs(FF))

#ordenamos los descriptores
idx = np.argsort(E)
idx = idx[::-1]

#graficamos
plt.figure()
plt.bar(np.arange(len(E)), E[idx])
plt.show()

cols = 5
sel = idx[0:cols]
```

Recuerde que en idx están los índices de los descriptores con mayor a menor información

- Características geométricas

Centro de Masa

Tamaño

Perímetro

Redondez

Momentos

Descriptores de Fourier

Elipse

Distancias

- Una vez calculados los descriptores, debemos seleccionar aquellos que provean más información (en caso que queramos modificar la forma de la región).

Recuerde que en este vector están los descriptores de Fourier

```
FF = np.fft.fft(CX)

#Energia
E = np.abs(FF)/np.sum(np.abs(FF))

idx = np.argsort(E)
idx = idx[::-1]

plt.figure()
plt.bar(np.arange(len(E)), E[idx])
plt.show()

cols = 5
sel = idx[0:cols]

iFF = np.zeros_like(FF)
iFF[sel] = FF[sel]
```

Vamos a seleccionar sólo los índices con mayor información

- Características geométricas

Centro de Masa

Tamaño

Perímetro

Redondez

Momentos

Descriptores de Fourier

Elipse

Distancias

- Una vez calculados los descriptores, debemos seleccionar aquellos que provean más información (en caso que queramos modificar la forma de la región).

```
idx = np.argsort(E)
idx = idx[::-1]

plt.figure()
plt.bar(np.arange(len(E)), E[idx])
plt.show()

cols = 5
sel = idx[0:cols]

iFF = np.zeros_like(FF)
iFF[sel] = FF[sel]

rF = np.fft.ifft(iFF)
```

La Transformada Inversa de Fourier nos permite transformar de la frecuencia al espacio

- Características geométricas

Centro de Masa

Tamaño

Perímetro

Redondez

Momentos

Descriptores de Fourier

Elipse

Distancias

- Se observa cómo a medida que incrementamos el número de descriptores, aumenta la precisión de los bordes.

```
new_x, new_y = np.real(rF), np.imag(rF)

plt.figure(dpi=150)
plt.imshow(binary, cmap='gray')
plt.plot(new_x, new_y, color='red')
plt.show()
```

Las funciones .real e .imag permiten obtener los valores de un arreglo con valores complejos

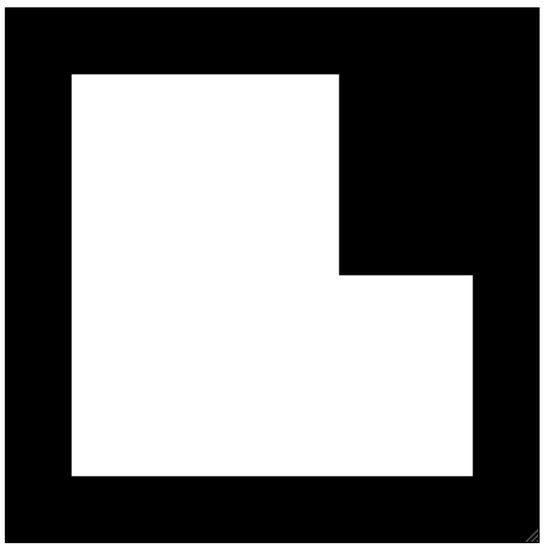
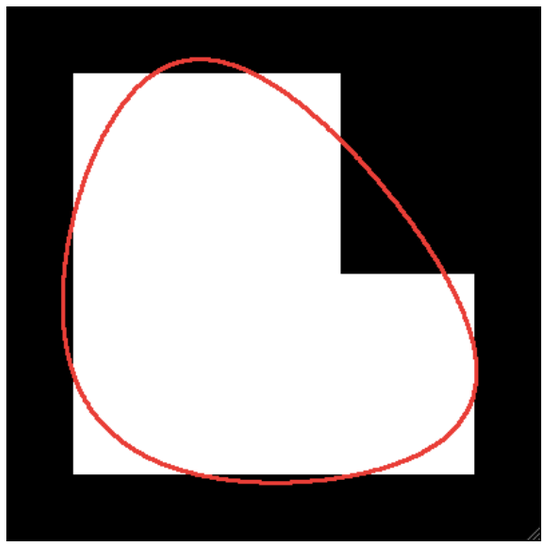
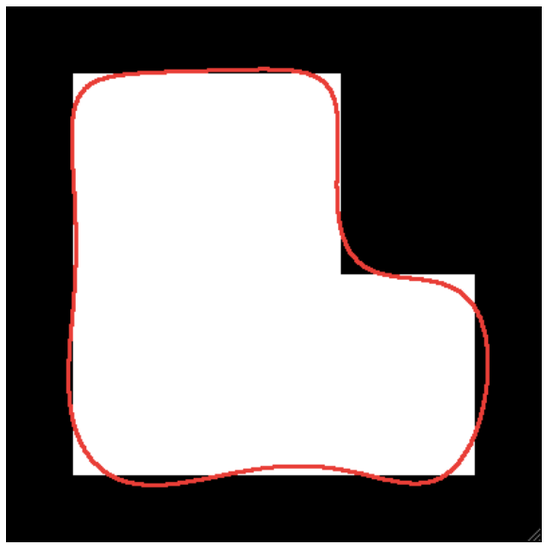


Imagen binaria



Línea Roja (5d)



Línea Roja (14d)

- Características geométricas

Centro de Masa

Tamaño

Perímetro

Redondes

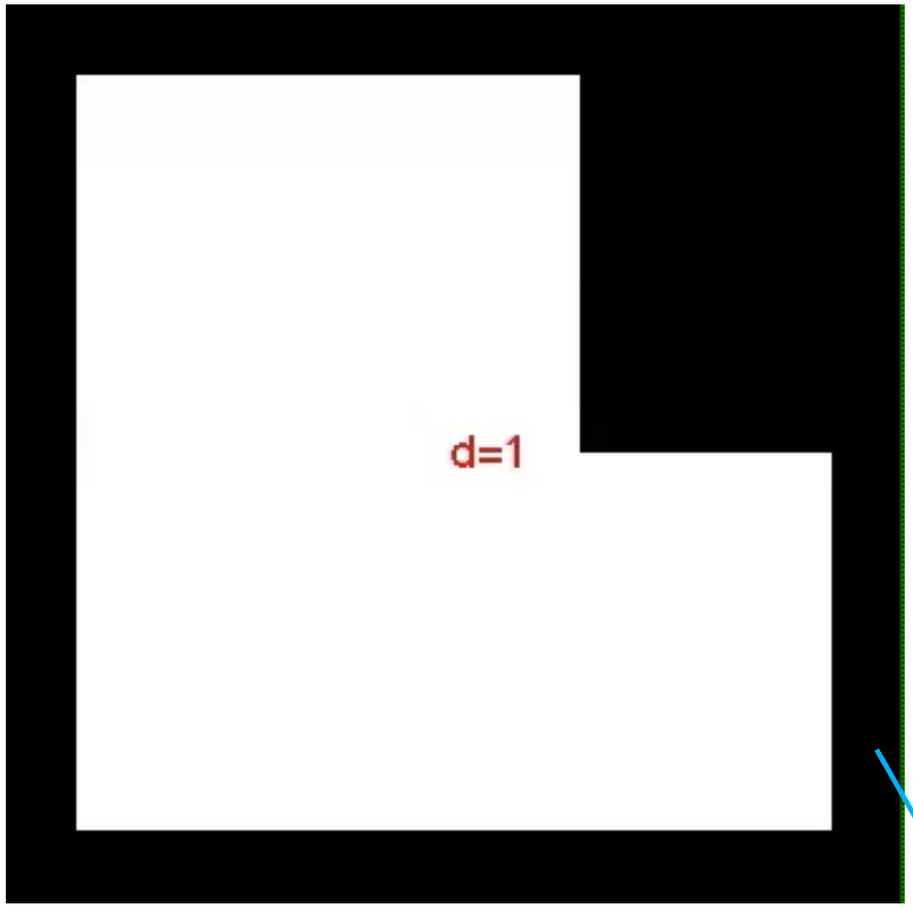
Momentos

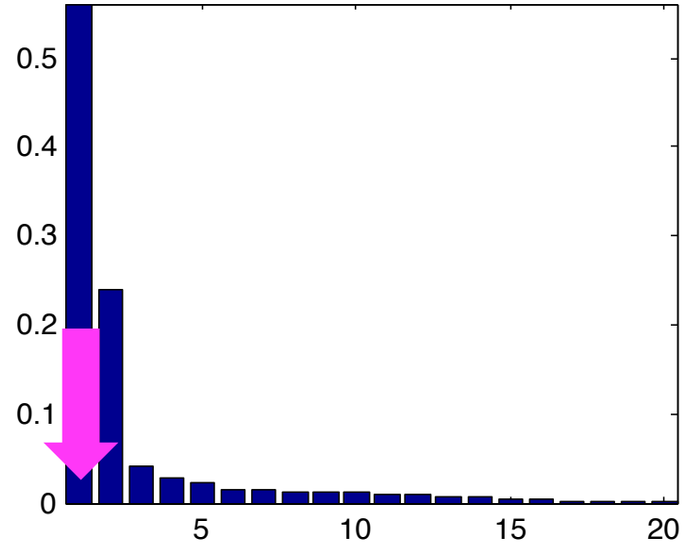
Descriptores de Fourier

Elipse

Distancias

- En el ejemplo, se observa cómo a medida que incrementamos el número de descriptores, aumenta la precisión de los bordes.





Observe cómo a medida que aumenta el número de descriptores empleados, la reconstrucción del borde aumenta su precisión

- Características geométricas

Centro de Masa

Tamaño

Perímetro

Redondez

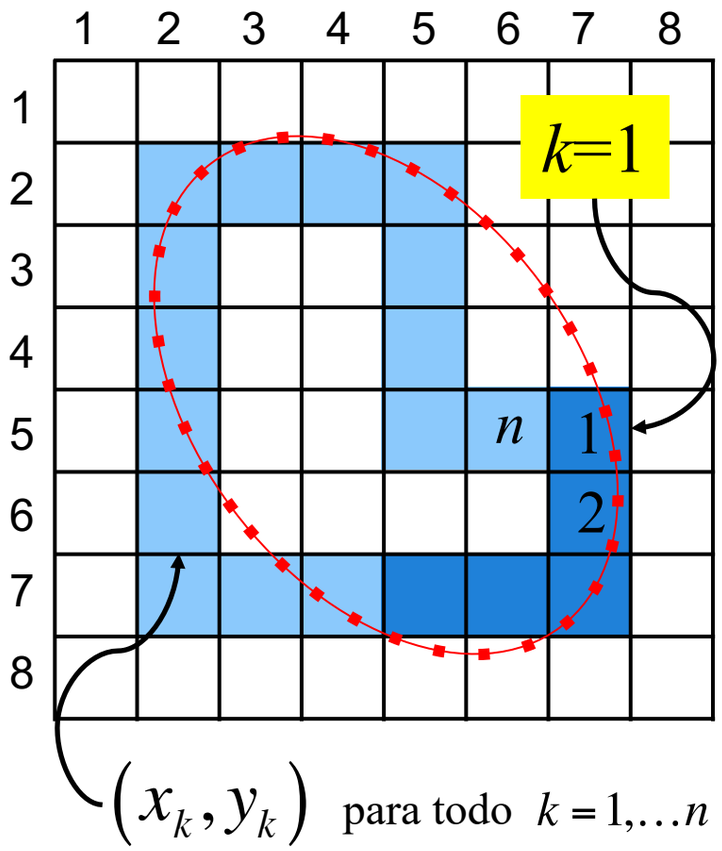
Momentos

Descriptores de Fourier

Elipse

Distancias

- Una forma para determinar la orientación de los objetos consiste en una estimación de la elipse.



Ecuación de la elipse

$$ax^2 + bxy + cy^2 + dx + ey + f = 0$$

restricción: $4ac - b^2 < 0$

Objetivo

Encontrar $\{a,b,c,d,e,f\}$ tal que:

$$\sum_{k=1}^{N}\left(ax^2 + bxy + cy^2 + dx + ey + f\right) \rightarrow \min$$

- Características geométricas

Centro de Masa

Tamaño

Perímetro

Redondez

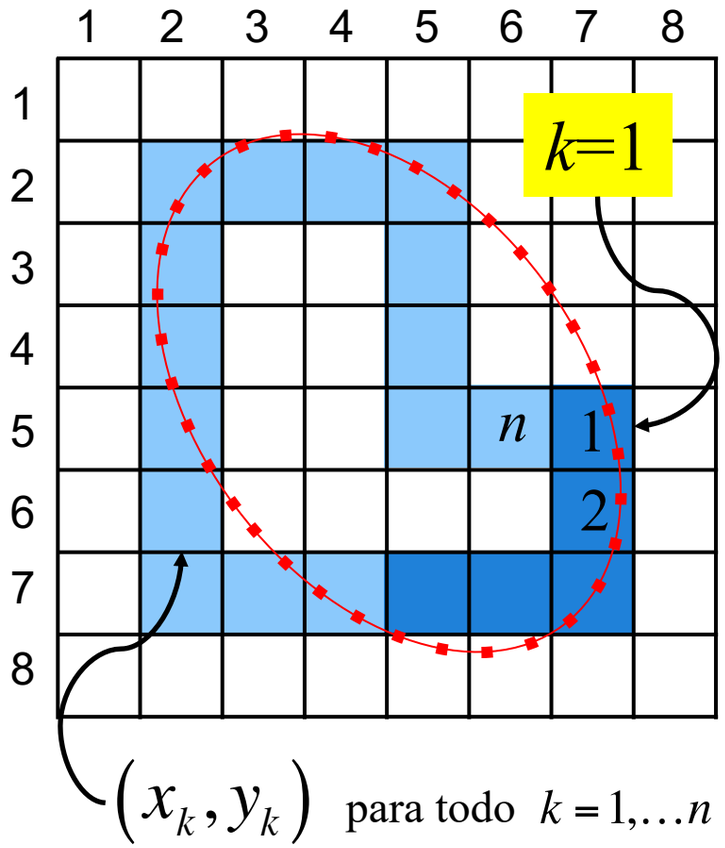
Momentos

Descriptores de Fourier

Elipse

Distancias

- Una forma para determinar la orientación de los objetos consiste en una estimación de la elipse.



Cada coordenada genera una fila en la matriz X

Método

$$\begin{bmatrix} x_1^2 & x_1y_1 & y_1^2 & x_1 & y_1 & 1 \\ x_2^2 & x_2y_2 & y_2^2 & x_2 & y_2 & 1 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ x_n^2 & x_ny_n & y_n^2 & x_n & y_n & 1 \end{bmatrix} \cdot \begin{bmatrix} a \\ b \\ c \\ d \\ e \\ f \end{bmatrix}$$

$\mathbf{X}$ $\mathbf{a}$

Objetivo

$$\|\mathbf{X} \cdot \mathbf{a}\| \rightarrow \min$$

donde $\mathbf{a} = \begin{bmatrix} a & b & c & d & e & f \end{bmatrix}^T$

- Características geométricas

Centro de Masa

Tamaño

Perímetro

Redondez

Momentos

Descriptores de Fourier

Elipse

Distancias

- Una forma para determinar la orientación de los objetos consiste en estimación de la elipse.

$$\begin{bmatrix} x_1^2 & x_1y_1 & y_1^2 & x_1 & y_1 & 1 \\ x_2^2 & x_2y_2 & y_2^2 & x_2 & y_2 & 1 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ x_n^2 & x_ny_n & y_n^2 & x_n & y_n & 1 \end{bmatrix} \cdot \begin{bmatrix} a \\ b \\ c \\ d \\ e \\ f \end{bmatrix}$$

$\mathbf{X}$ $\qquad$ $\mathbf{a}$

El algoritmo debe construir la matriz $\mathbf{X}$ a partir de las coordenadas del perímetro.

Fitzgibbon, A.W., Pilu, M and Fischer, R. B.: Direct least squares fitting of ellipses. In *Proc. of the 13th International Conference on Pattern Recognition*, pages 253–257, Vienna, September 1996

Solución matemática de Fitzgibbon (1996)

suponemos que $4ac - b^2 = 1$

$$\min_{\mathbf{a}} \|\mathbf{X} \cdot \mathbf{a}\| \quad \text{sujeto a} \quad \mathbf{a}^T\mathbf{C}\mathbf{a} = 1$$

donde $\mathbf{C} = \begin{bmatrix} 0 & 0 & 2 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 & 0 & 0 \\ 2 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$

aplicando operadores de Lagrange $\mathbf{Sa} = \lambda\mathbf{Ca}$

donde $\mathbf{S} = \mathbf{D}'\mathbf{D}$

la solución queda en el menor valor propio positivo

$$[\mathbf{vec}, \mathbf{val}] = eig(\mathbf{S}, \mathbf{C})$$

- Características geométricas

Centro de Masa

Tamaño

Perímetro

Redondez

Momentos

Descriptores de Fourier

Elipse

Distancias

- Una forma para determinar la orientación de los objetos consiste en estimación de la elipse.

parámetros

$$\mathbf{X}\cdot\mathbf{a} = \begin{bmatrix} x_1^2 & x_1y_1 & y_1^2 & x_1 & y_1 & 1 \\ x_2^2 & x_2y_2 & y_2^2 & x_2 & y_2 & 1 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ x_n^2 & x_ny_n & y_n^2 & x_n & y_n & 1 \end{bmatrix} \cdot \begin{bmatrix} a \\ b \\ c \\ d \\ e \\ f \end{bmatrix}$$

```
def fitzgibbon(x, y):
    n = x.shape[0]                                  # puntos de la frontera
    X = [x**2, x*y, y**2, x, y, np.ones((n,1))] # matriz X
    X = np.hstack(X)
    S = np.matmul(X.T,X)                            # matriz dispersión
    C = np.zeros((6,6))
    C[0,2]=2; C[1,1]=-1; C[2,0]=2;                  # matriz restricción

    E, V = np.linalg.eig(np.dot(np.linalg.inv(S),C))
    idx = np.argwhere(E>0 & ~np.isinf(E))
    params = V[:,idx]                               # valor propio
    return params
```

- Características geométricas

Centro de Masa

Tamaño

Perímetro

Redondez

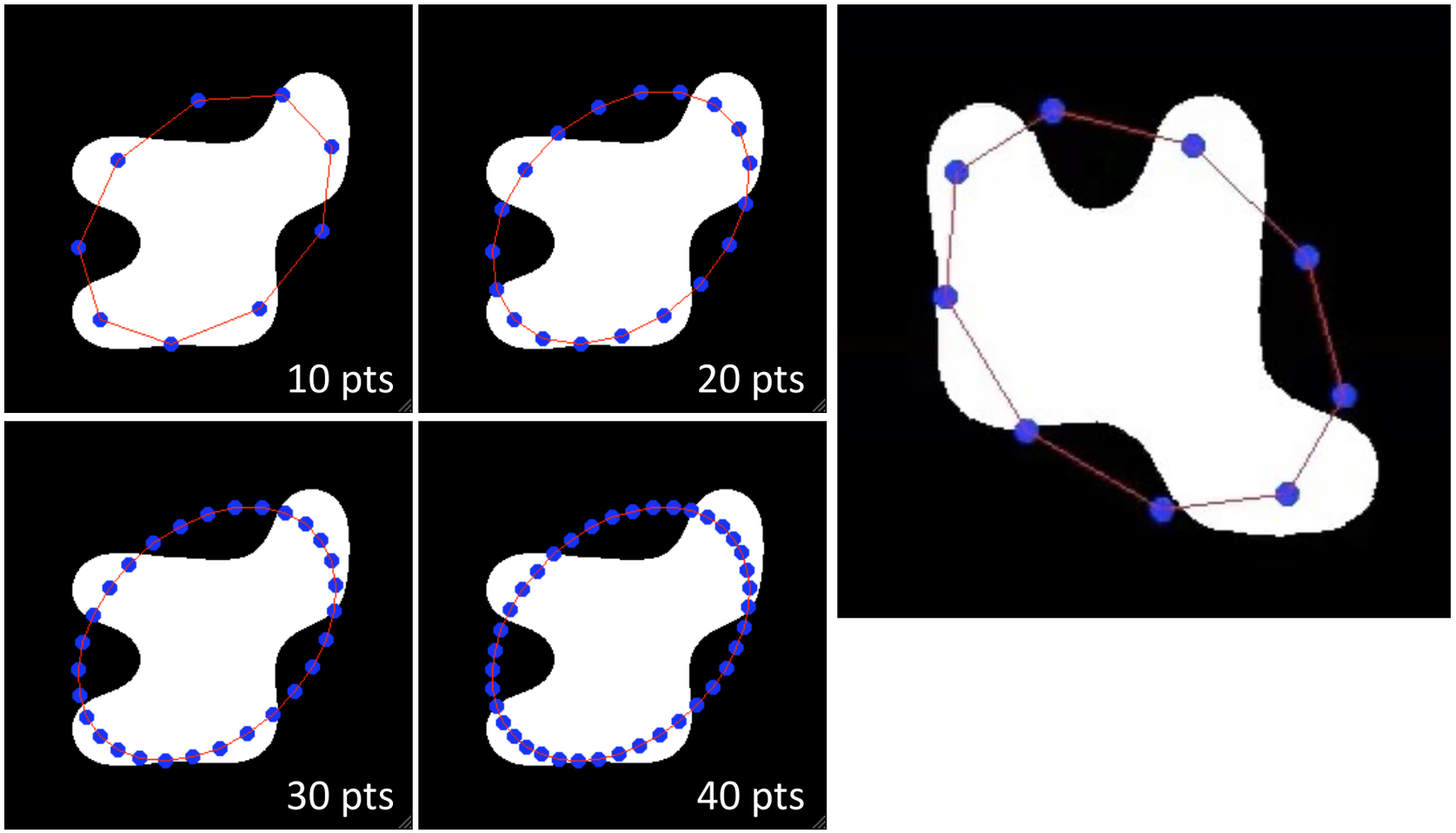
Momentos

Descriptores de Fourier

Elipse

Distancias

- Una forma para determinar la orientación de los objetos consiste en estimación de la elipse.



- Características geométricas

Centro de Masa

Tamaño

Perímetro

Redondez

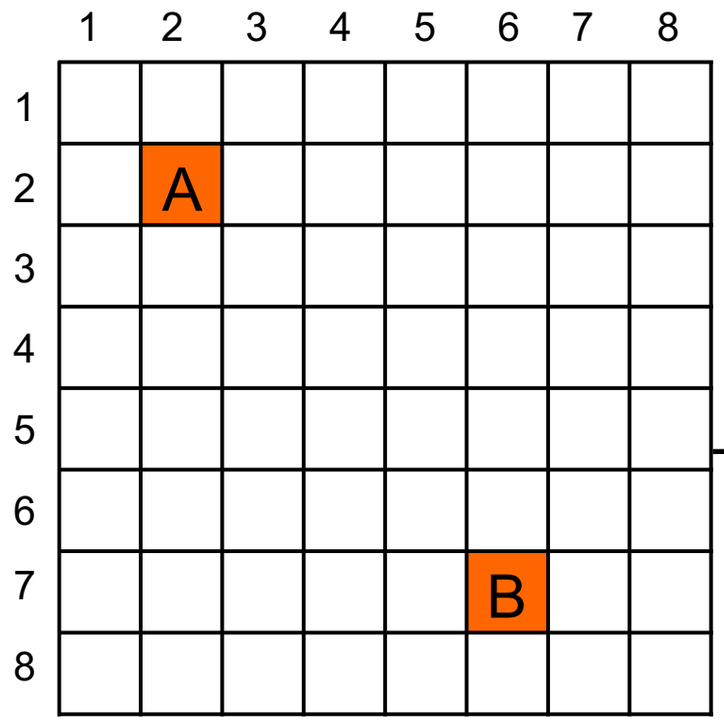
Momentos

Descriptores de Fourier

Elipse

Distancias

- La distancia entre dos puntos puede ser medida de diversas formas. En especial consideración tiene la relación discreta de los píxeles en la imagen.



¿Cómo calculamos la distancia entre A y B?

- Características geométricas

Centro de Masa

Tamaño

Perímetro

Redondez

Momentos

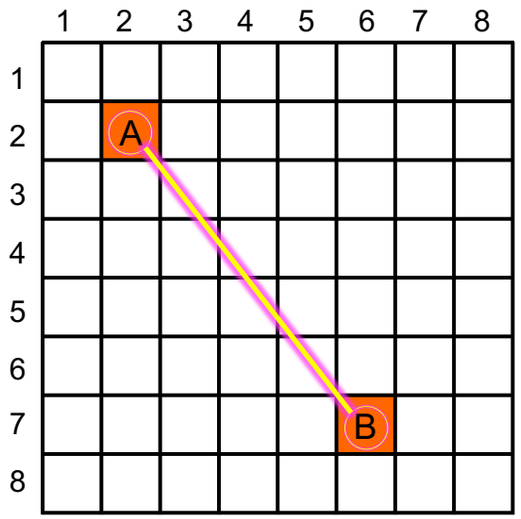
Descriptores de Fourier

Elipse

Distancias

- La distancia siempre es independiente de la ruta ya sólo considera la posición de los píxeles (el ejemplo es sólo ilustrativo).

Distancia Euclidiana

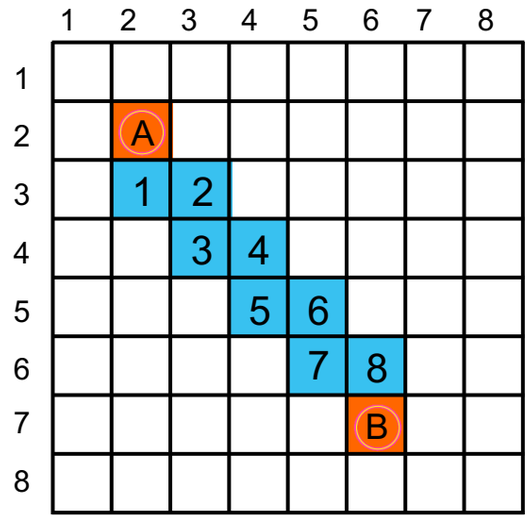


$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

$$d = \sqrt{(7-2)^2 + (6-2)^2}$$

$$d = 6.4$$

Distancia City Block
(conectividad 4)

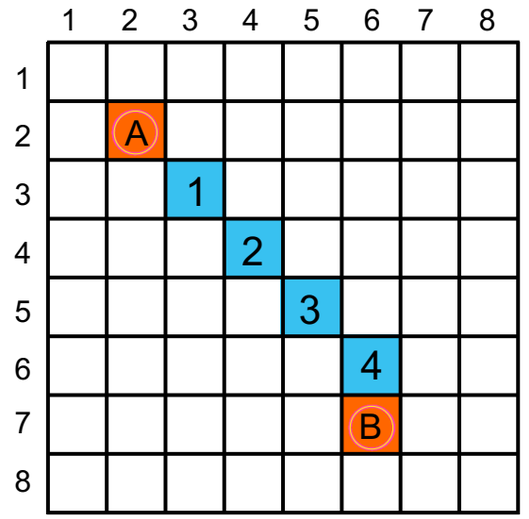


$$d = |x_1 - x_2| + |y_1 - y_2|$$

$$d = |2-7| + |6-2|$$

$$d = 9$$

Distancia Chessboard
(conectividad 8)



$$d = \max\left(|x_1 - x_2|, |y_1 - y_2|\right)$$

$$d = \max\left(|2-7|, |6-2|\right)$$

$$d = \max(5,4) = 5$$

- Características geométricas

Centro de Masa

Tamaño

Perímetro

Redondez

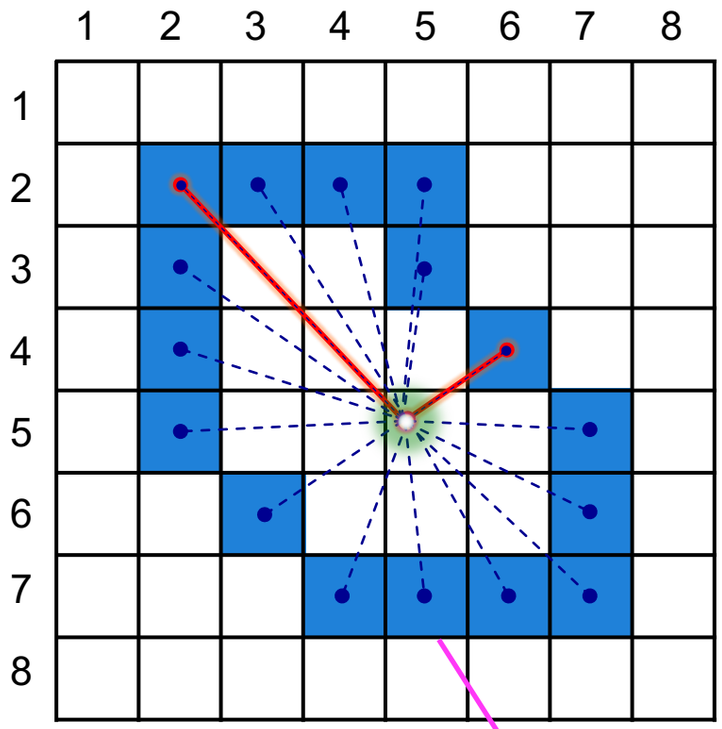
Momentos

Descriptores de Fourier

Elipse

Distancias

- Otro descriptor empleado es el cálculo de la distancia del centro de masa a la frontera de la región.



Calculamos el centro con los momentos

Definiciones

$d_{\min}$: distancia mínima

$d_{\max}$: distancia máxima

d_{mean} : distancia promedio

No son invariantes

Descriptores invariantes (normalizados

$$\left\{ \frac{d_{\max}}{d_{\min}}, \frac{d_{\max}}{d_{mean}}, \frac{d_{\min}}{d_{mean}} \right\}$$

- Características geométricas

Centro de Masa

Tamaño

Perímetro

Redondez

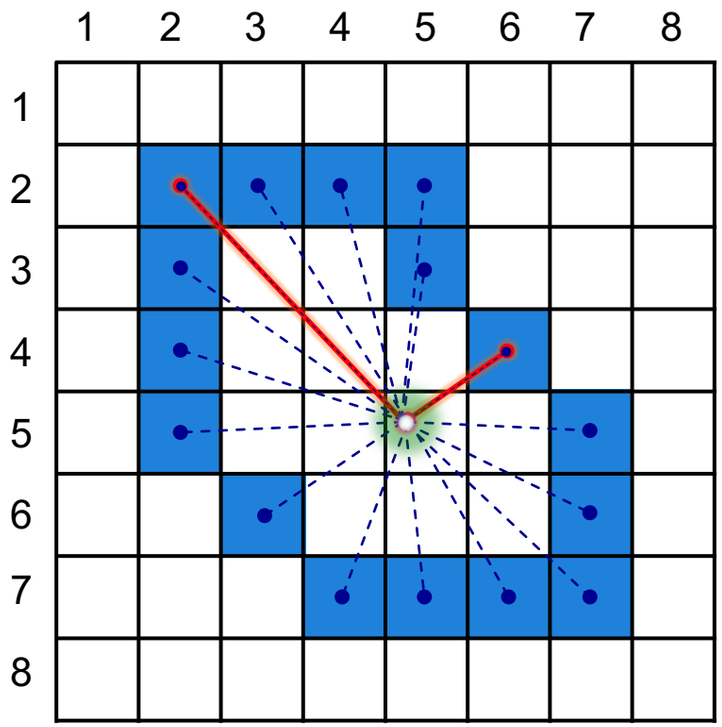
Momentos

Descriptores de Fourier

Elipse

Distancias

- Otro descriptor empleado es el cálculo de la distancia del centro de masa a la frontera de la región.




```
def distancia(image):

    cnts, _ = cv2.findContours(image,
                  cv2.RETR_EXTERNAL,
                  cv2.CHAIN_APPROX_NONE)

    contornos = np.vstack(cnts[0])
    x , y = zip(*contornos)
    x = np.array(x).reshape(-1,1)
    y = np.array(y).reshape(-1,1)

    [ci, cj] = centro_masa(image)

    dd = np.sqrt((x-ci)**2+(y-cj)**2)

    #descriptores normalizados
    dmean = np.mean(dd)
    dmax  = np.max(dd)/dmean
    dmin  = np.min(dd)/dmean
    delta = dmax/dmin

    return dmax, dmin, delta
```

centro de masa

- Características geométricas

Centro de Masa

Tamaño

Perímetro

Redondez

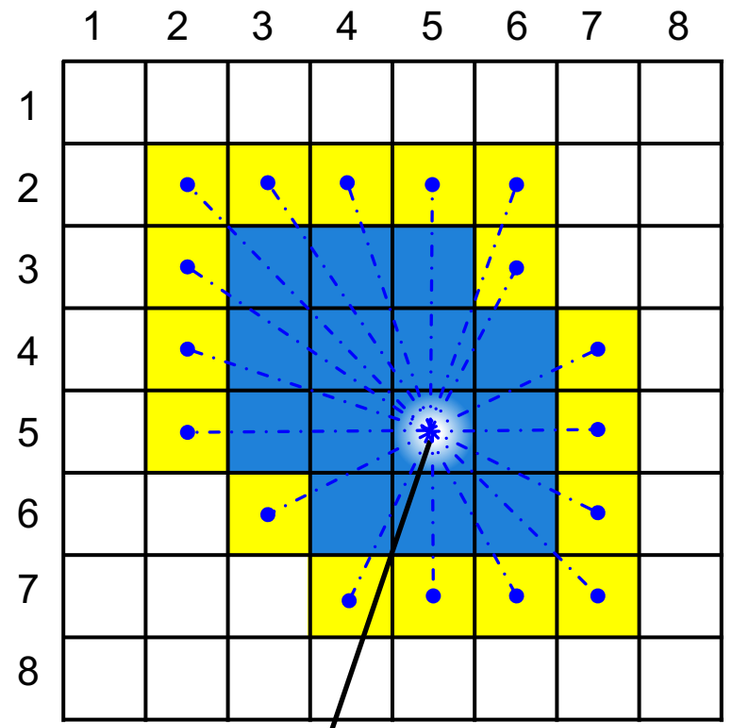
Momentos

Descriptores de Fourier

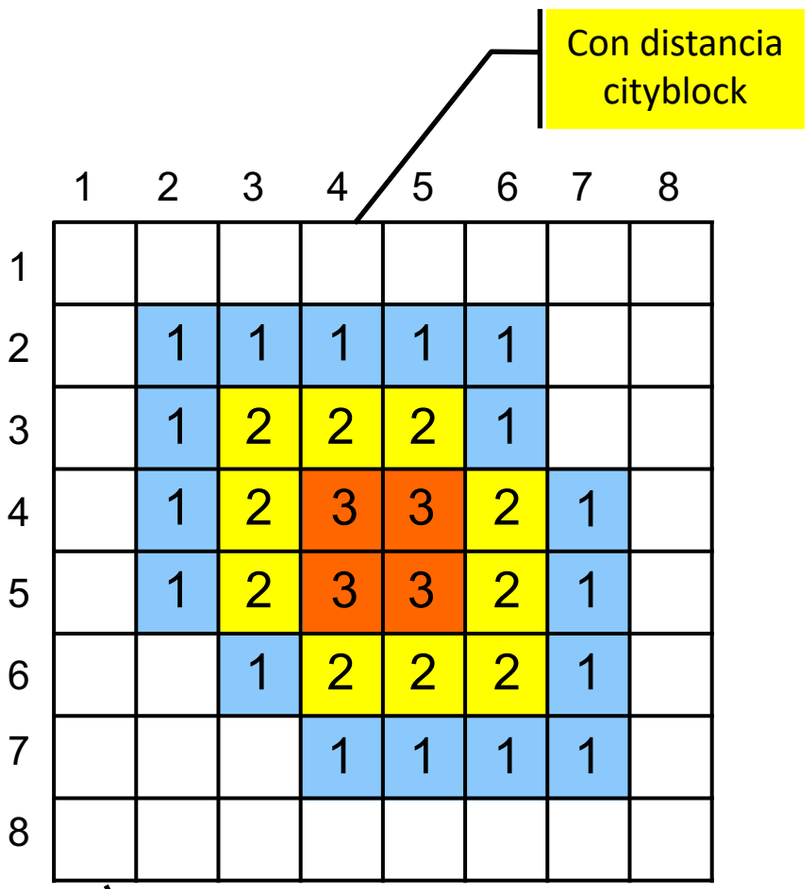
Elipse

Distancias

- Otra medida consiste en determinar la distancia de todos los píxeles de la región a la frontera de ésta.



¿Cuál es la menor distancia de éste píxel al borde?



Por cada píxel, calculamos la mínima distancia al fondo

- Características geométricas

Centro de Masa

Tamaño

Perímetro

Redondez

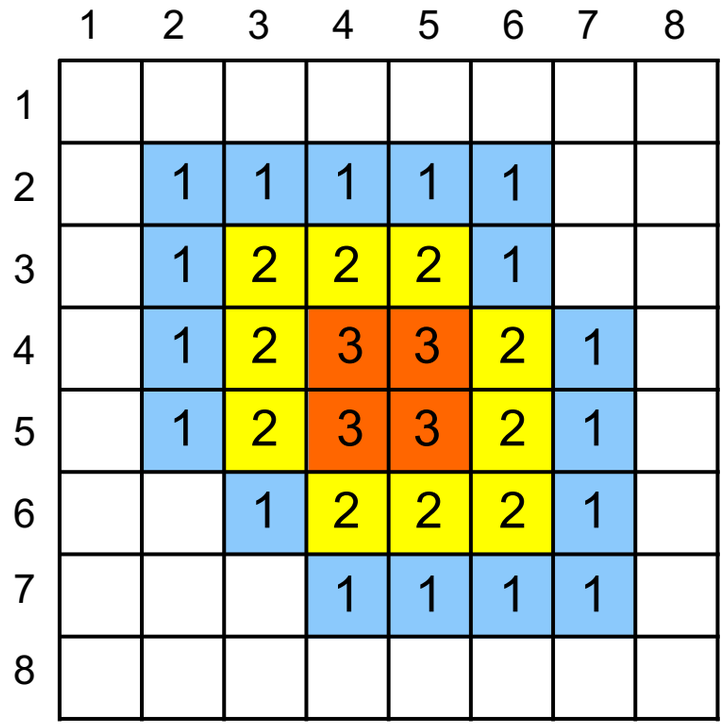
Momentos

Descriptores de Fourier

Elipse

Distancias

- Otra medida consiste en determinar la distancia de todos los píxeles de la región al borde de ésta



Definiciones

A : área de la región

$\beta = \frac{1}{A}\sum_{i \in R} d_i$: distancia promedio

Descriptor

$f = \frac{A}{\beta^2}$: complejidad de forma

- Características geométricas

Centro de Masa

Tamaño

Perímetro

Redondez

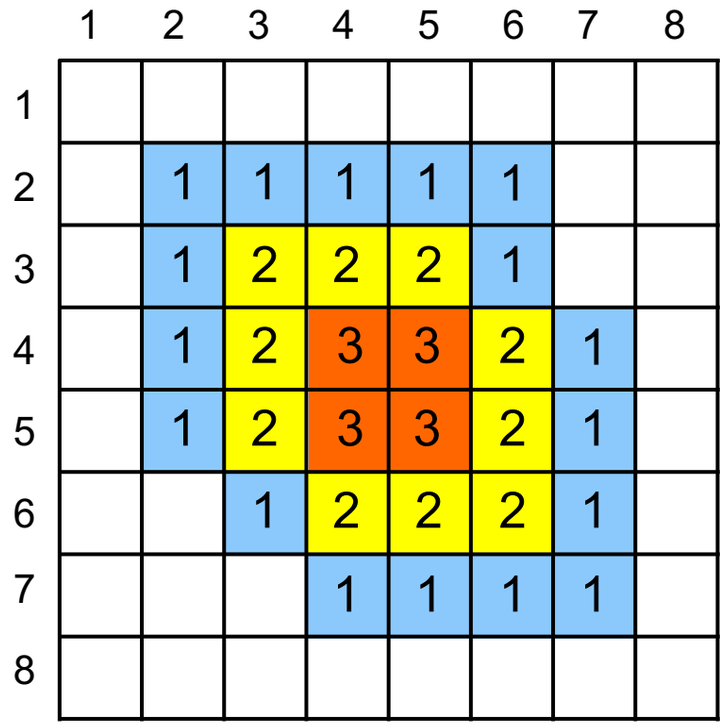
Momentos

Descriptores de Fourier

Elipse

Distancias

- Otra medida consiste en determinar la distancia de todos los píxeles de la región al borde de ésta



Definiciones

$A = 31$

$\beta = \frac{1}{31}(17 * 1 + 10 * 2 + 4 * 3) = 1.58$

Complejidad de forma

$$f = \frac{A}{\beta^2} = \frac{31}{1.58^2} = 12.40$$

- Características geométricas

Centro de Masa

Tamaño

Perímetro

Redondez

Momentos

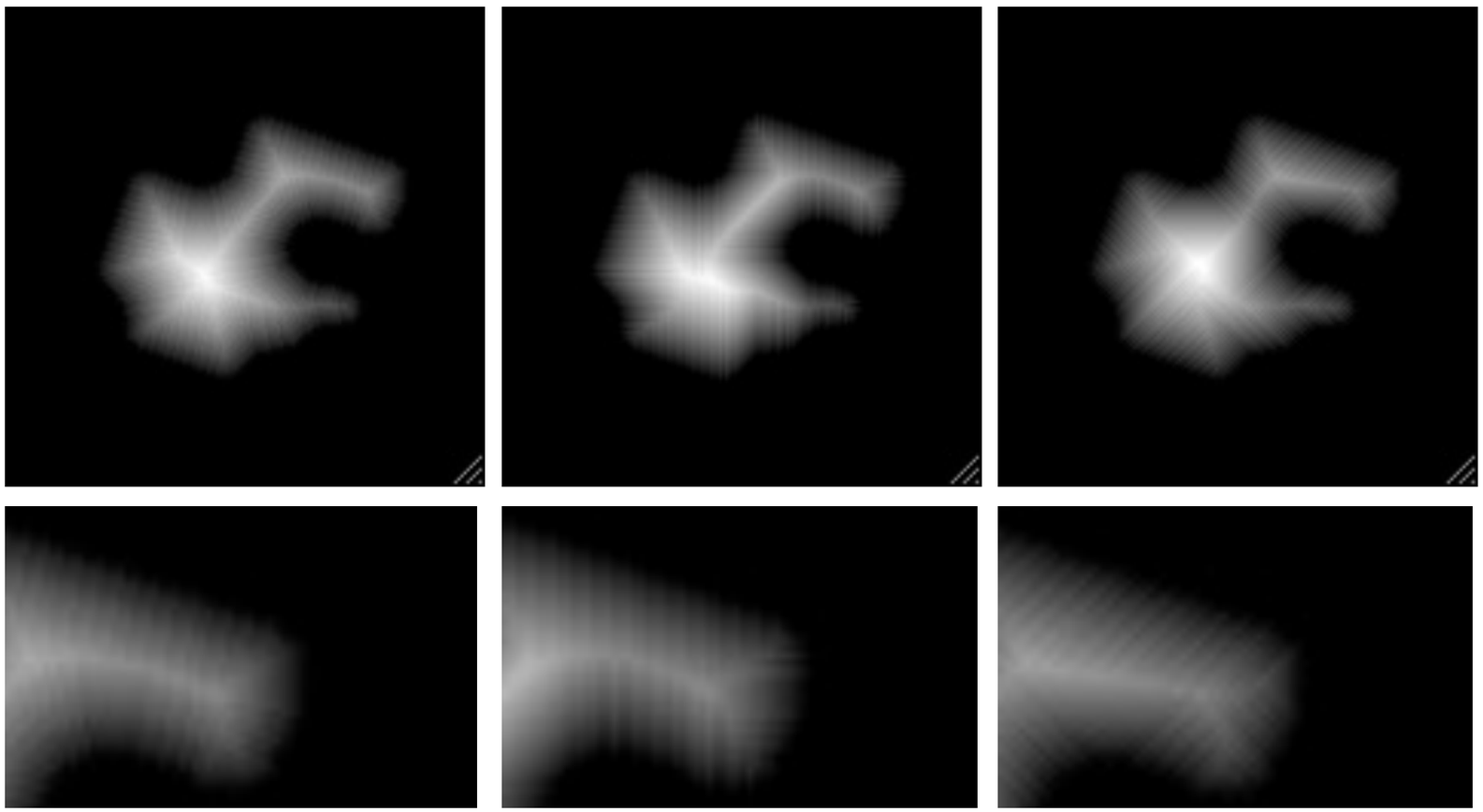
Descriptores de Fourier

Elipse

Distancias

- Otra medida consiste en determinar la distancia de todos los píxeles de la región al borde de ésta.

Distancia Euclidiana | Distancia City Block | Distancia Chessboard

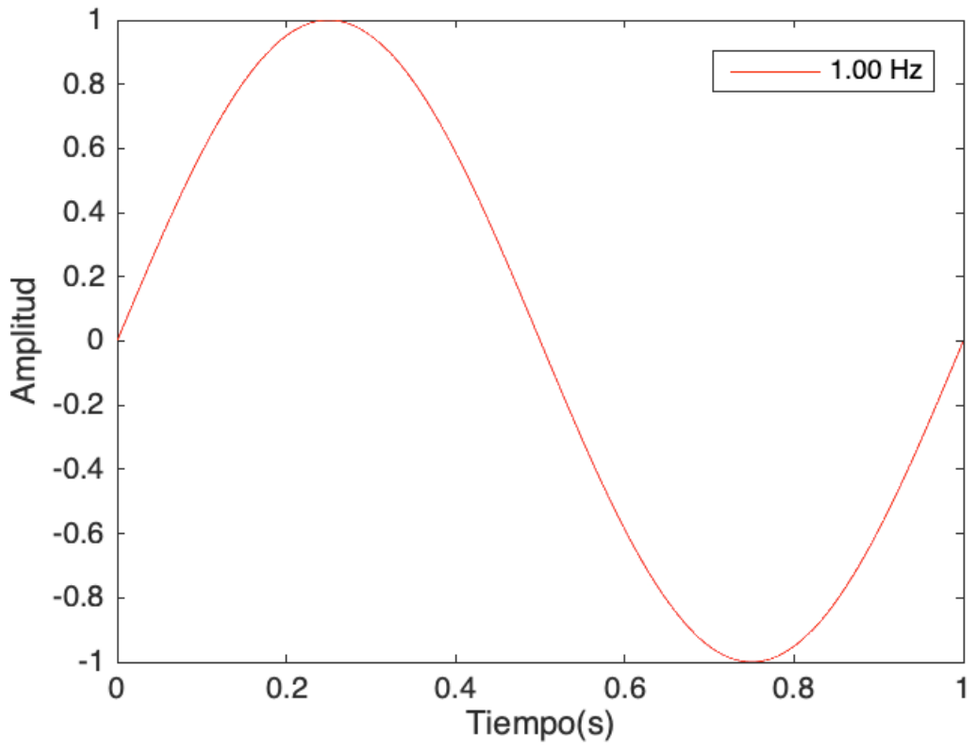


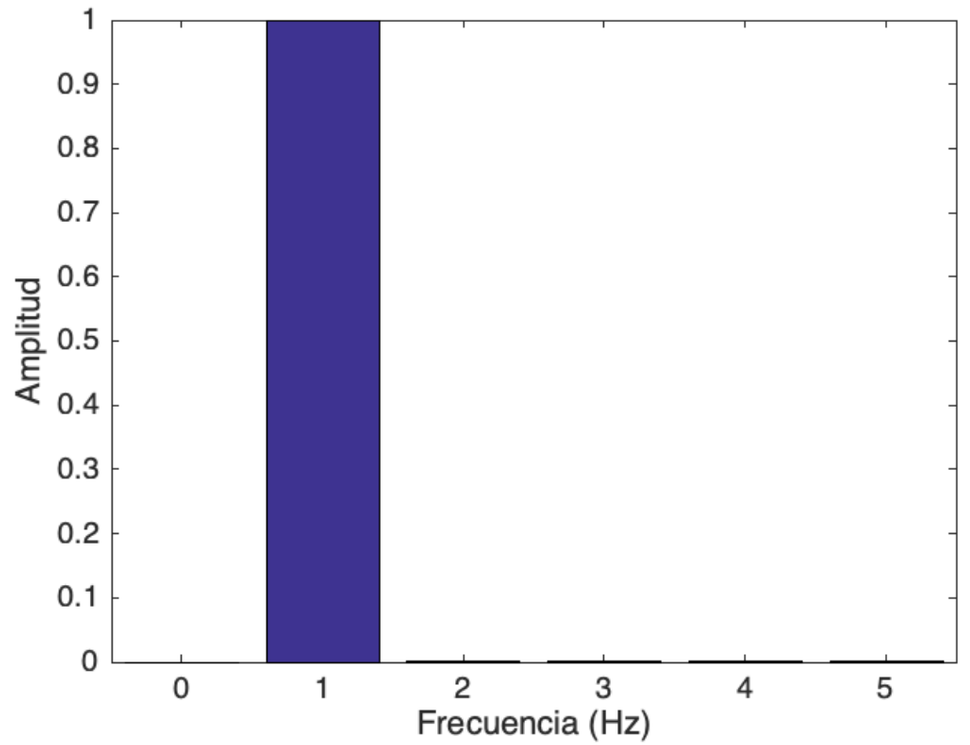
- Anexo
- Transformada de Fourier



Ciclos por segundo

- Recordemos algunos puntos de la señal periódica





Frecuencia angular

$$w = 2\pi f$$

Frecuencia (Hz)

Frecuencia (Hz)

$$f = \frac{1}{T}$$

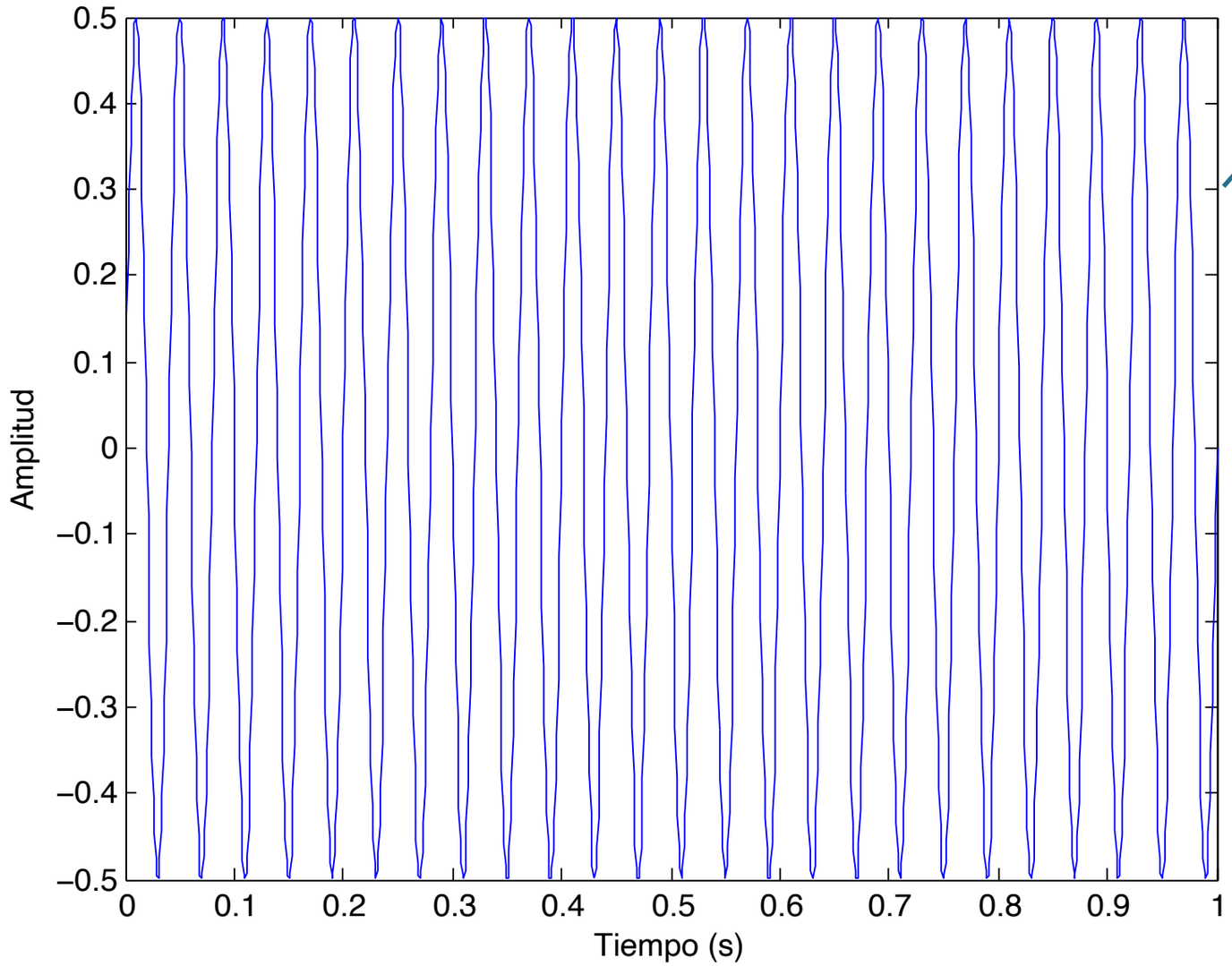
Tiempo

Observe cómo se modifica la frecuencia a medida que aumenta los ciclos por segundo.



Ejemplo 1

- Suponga que ahora no sabes qué frecuencia tiene una señal.



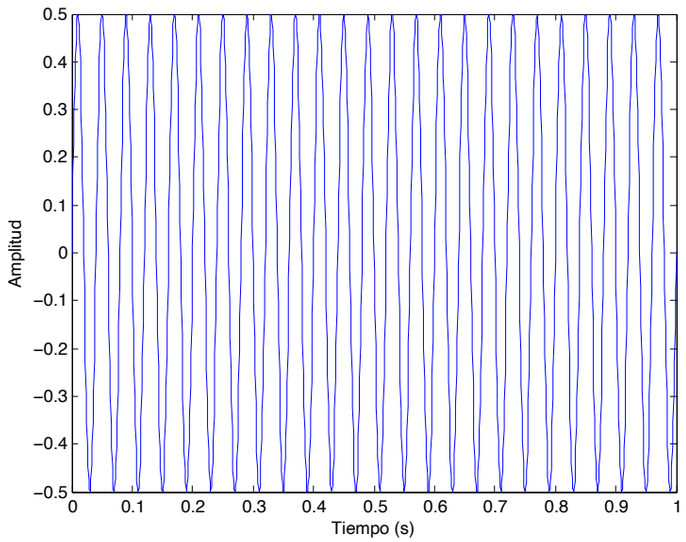
Qué frecuencia tiene la señal que muestra la figura (suponga que desconoce cómo la determinamos)

¿cómo la determinamos?



Ejemplo 1

- Primero: generemos el tiempo de la señal; en este ejemplo espaciada a un 1 milisegundo.



Frecuencia

o ciclos por segundo
se mide en Hertz

Significa 1000 ciclos en 1 segundo

$$f = \frac{1}{T} \Rightarrow f = \frac{1}{0.001s} = 1000\,Hz$$

Tiempo

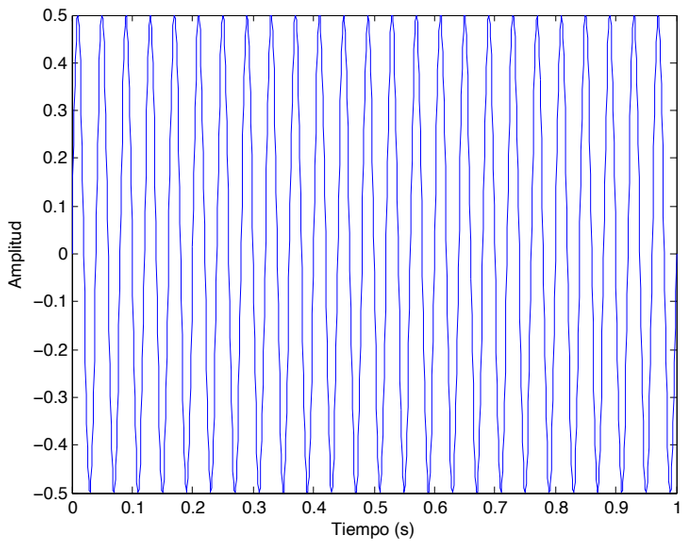
```
import numpy as np
import matplotlib.pyplot as plt

t= np.linspace(0,1, num=1000)
```

El comando **`linspace`**, genera una separación lineal entre dos valores separados por una cantidad fija de valores

Ejemplo 1

- Segundo: generemos una señal de 25Hz y 0.5 de amplitud



Esto es la frecuencia angular, es decir, ciclos por segundo. Se mide en *rad/s*

$$w = 2\pi f$$

```
import numpy as np
import matplotlib.pyplot as plt

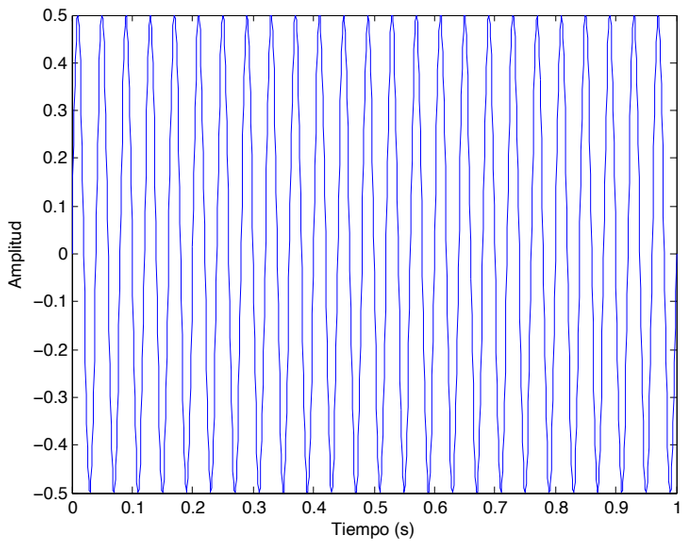
t= np.linspace(0,1, num=1000)
y = 0.5*np.sin(2*np.pi*t*25)
```

Inventemos una señal sinusoidal de 25 Hz y 0.5 de amplitud

Esto significa que generamos 25 ciclos en un segundo con una amplitud de 0.5.

Ejemplo 1

- Tercero: generemos una señal de 25Hz y 0.5 de amplitud




```
import numpy as np
import matplotlib.pyplot as plt

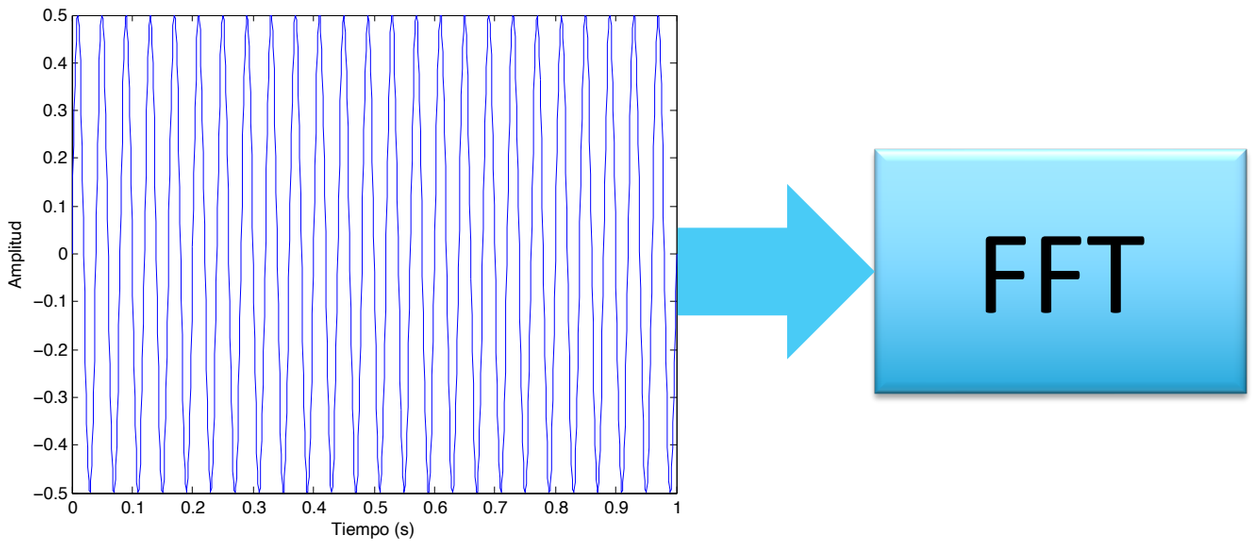
t= np.linspace(0,1, num=1000)
y = 0.5*np.sin(2*np.pi*t*25)

plt.plot(t, y)
plt.show()
```

Ploteamos la señal
(Tiempo vs Amplitud)

Ejemplo 1

- Cuarto: Calculemos la Transformada de Fourier de la señal original.




```
import numpy as np
import matplotlib.pyplot as plt

t= np.linspace(0,1, num=1000)
y = 0.5*np.sin(2*np.pi*t*25)

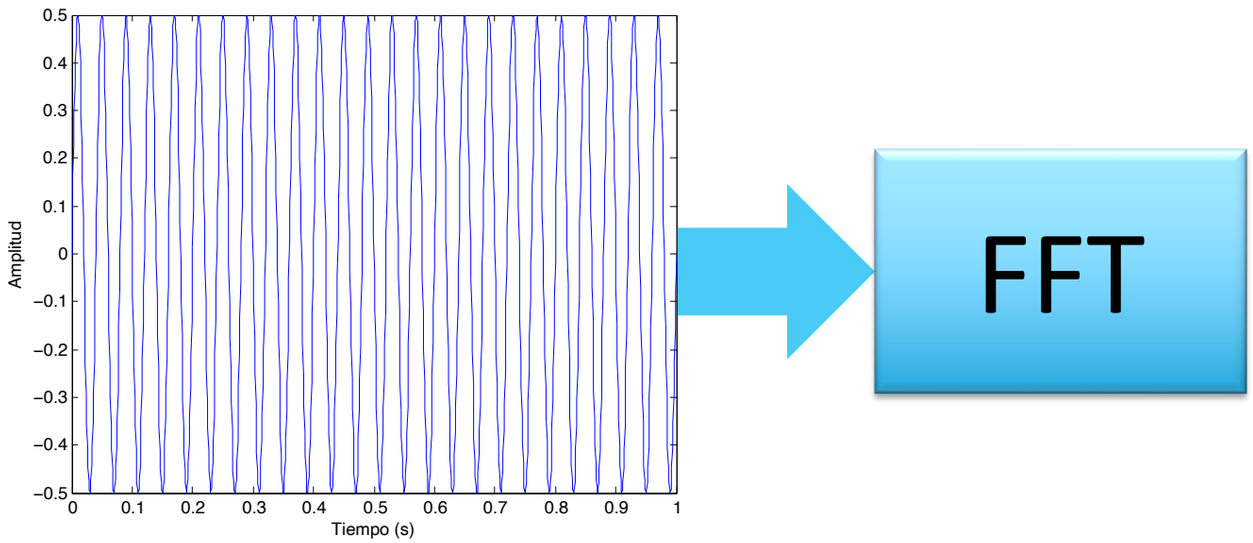
plt.plot(t, y)
plt.show()

F = np.fft.fft(y)
```

Aplicamos la Transformada Discreta de Fourier (DFT) (ultra rápida)

Ejemplo 1

- Quinto: Ploteamos el tiempo versus el valor absoluto de la señal



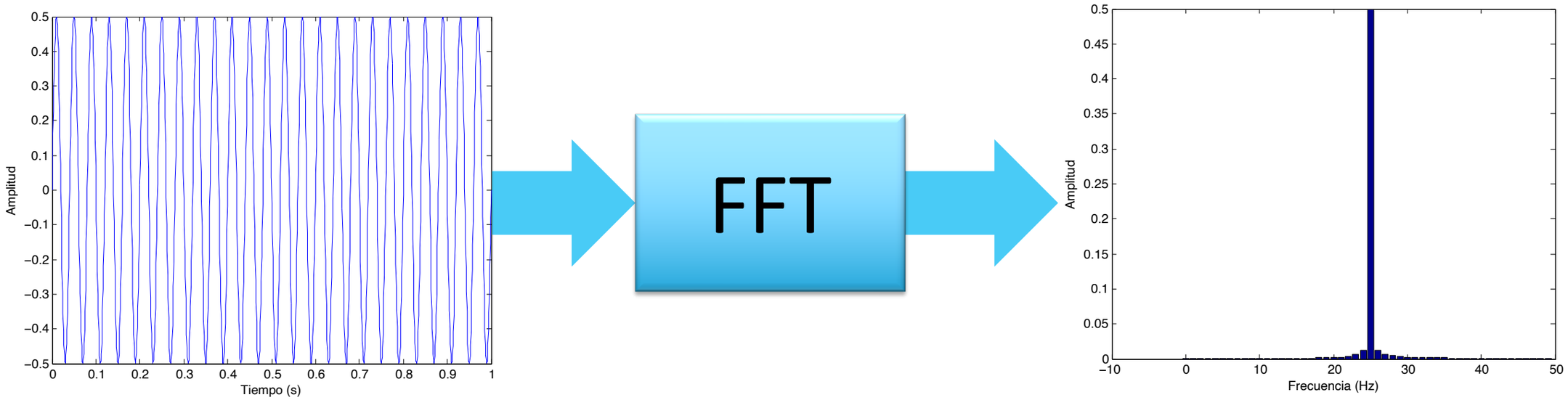

```
t= np.linspace(0,1, num=1000)
y = 0.5*np.sin(2*np.pi*t*25)

F = np.fft.fft(y)
N = len(F)
```

Calculamos el número de puntos de la muestra

Ejemplo 1

- Quinto: Ploteamos el tiempo versus el valor absoluto de la señal




```
t= np.linspace(0,1, num=1000)
y = 0.5*np.sin(2*np.pi*t*25)

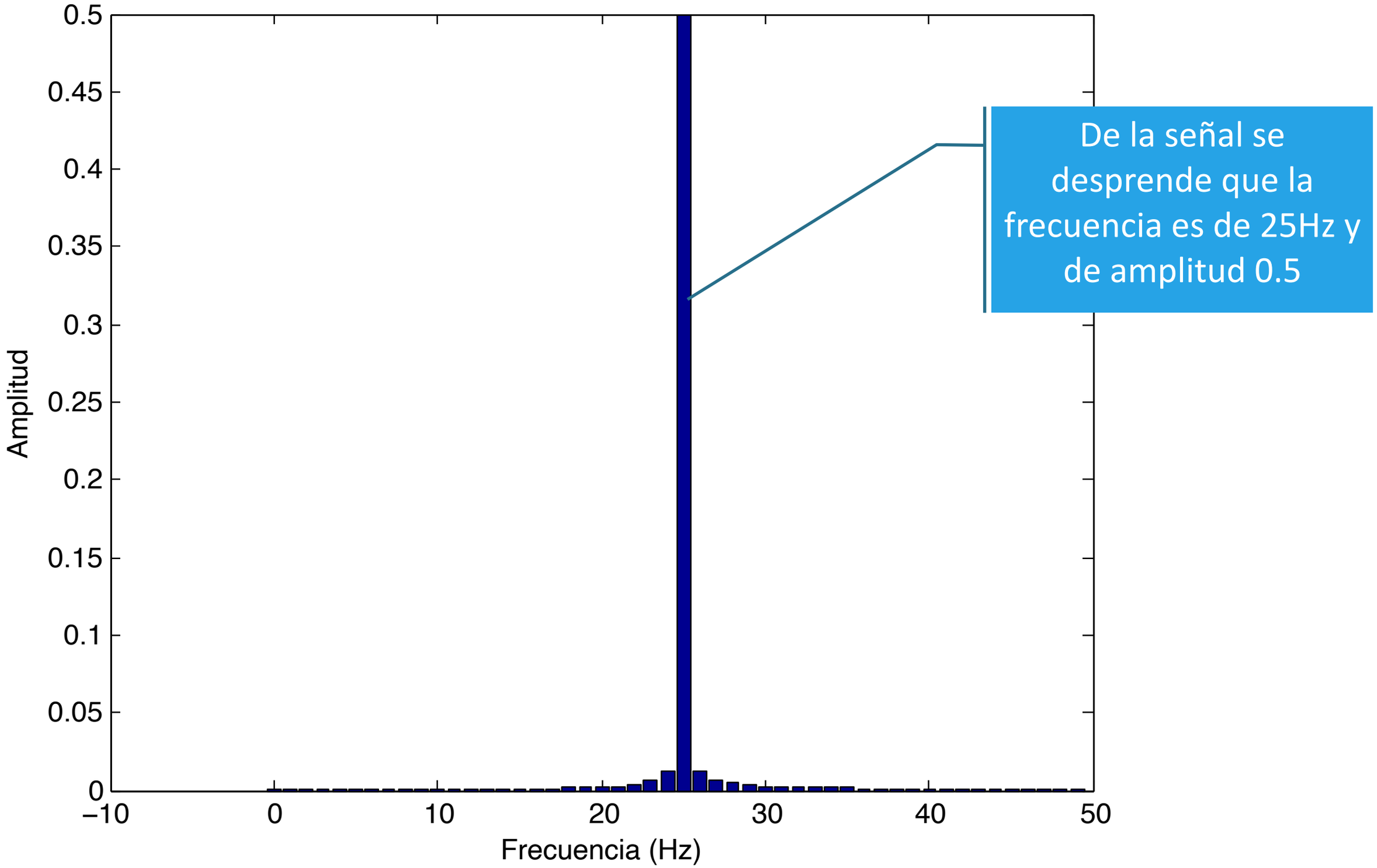
F = np.fft.fft(y)
N = len(F)

p= np.linspace(0, 49, dtype=int, num=50)

plt.bar(x=p, height=abs(F[p])/N*2)
plt.show()
```

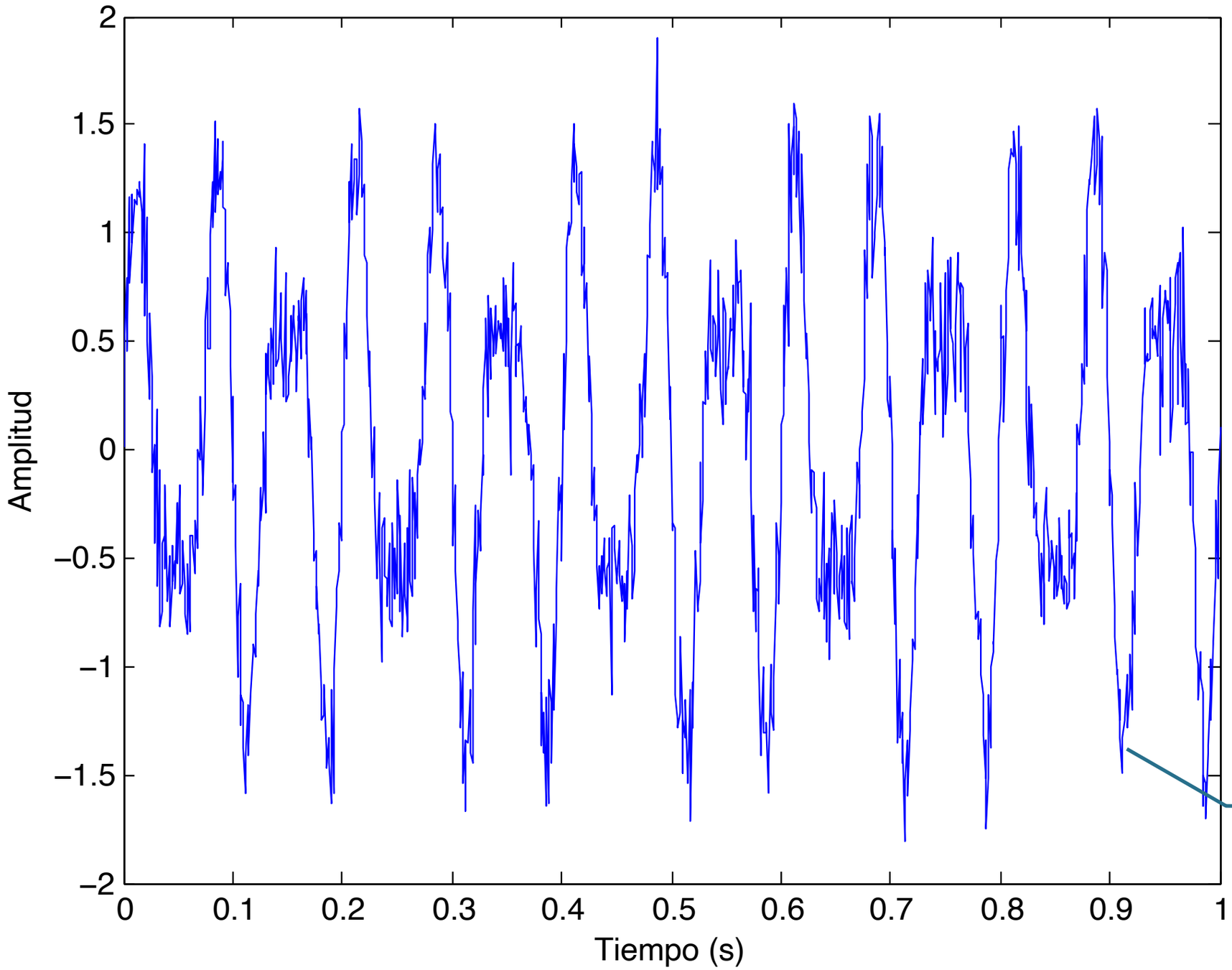
Calculamos el valor absoluto de la señal para obtener la amplitud. (en este caso desde 1 a 50Hz)

- Más en detalle



Ejemplo 2

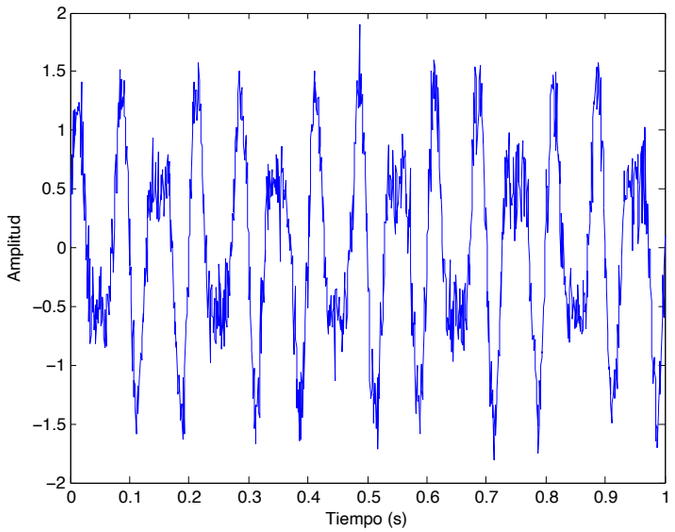
- Compliquemos las cosas, ¿qué frecuencia tiene la siguiente señal?

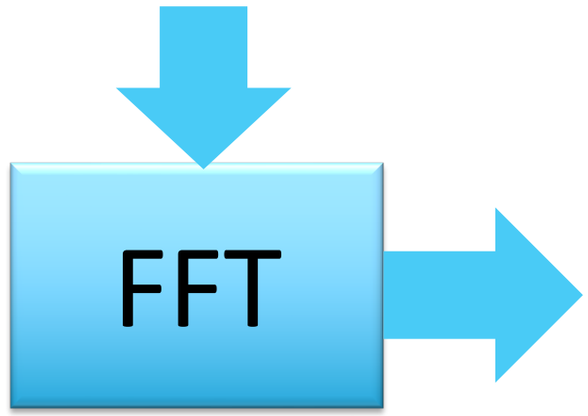


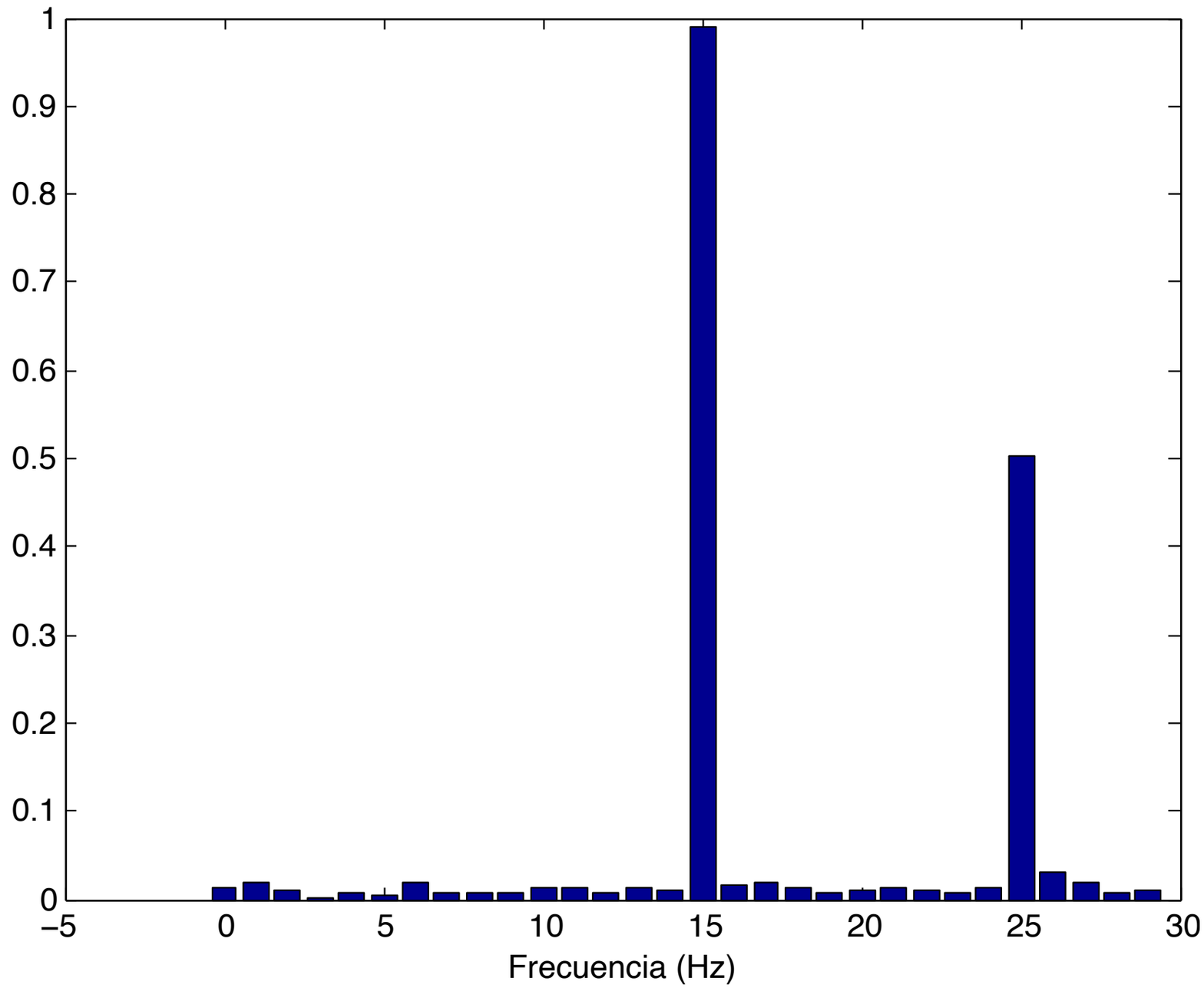
Si la señal además tiene ruido

Ejemplo 2

- Ahora es más claro que la señal es aproximadamente

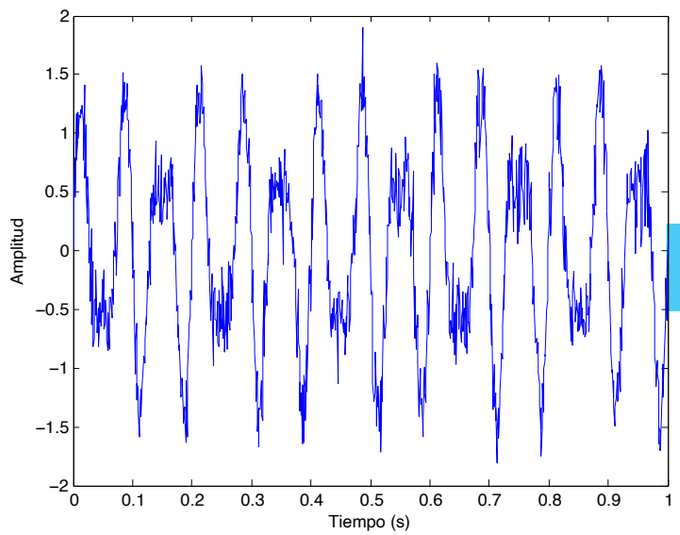




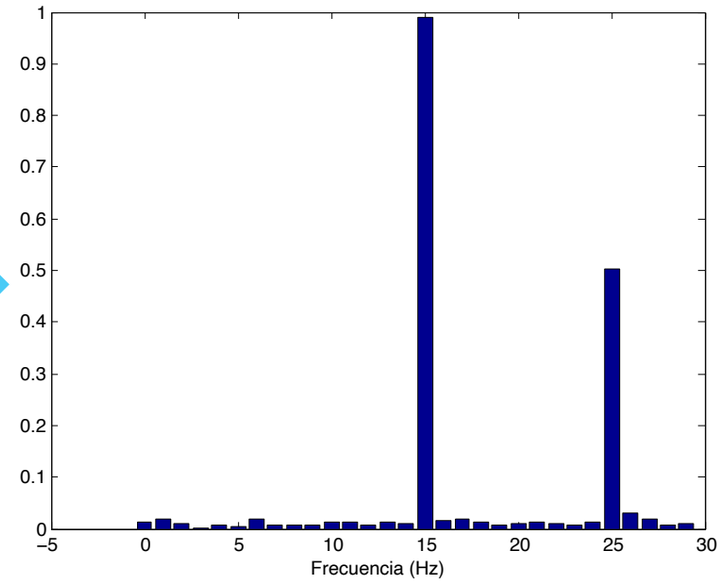


Ejemplo 2

- La señal es aproximadamente: $0.5\sin(2\pi \cdot 25) + \sin\left(2\pi \cdot 15\right)$



FFT




```
t= np.linspace(0,1,num =1000) #tiempo
R = np.random.random(1000)

y = 0.5*np.sin(2*np.pi*25*t)+np.sin(2*np.pi*15*t)+0.2*R
```

Aunque la señal tiene ruido, aún es posible encontrar las frecuencias correctas

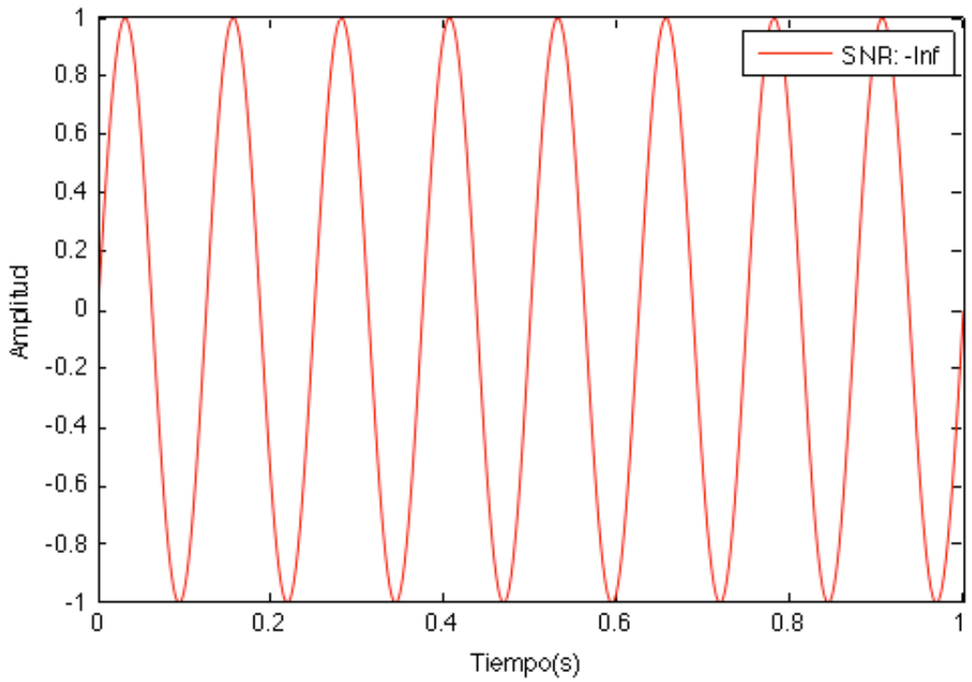
Buscando las señales

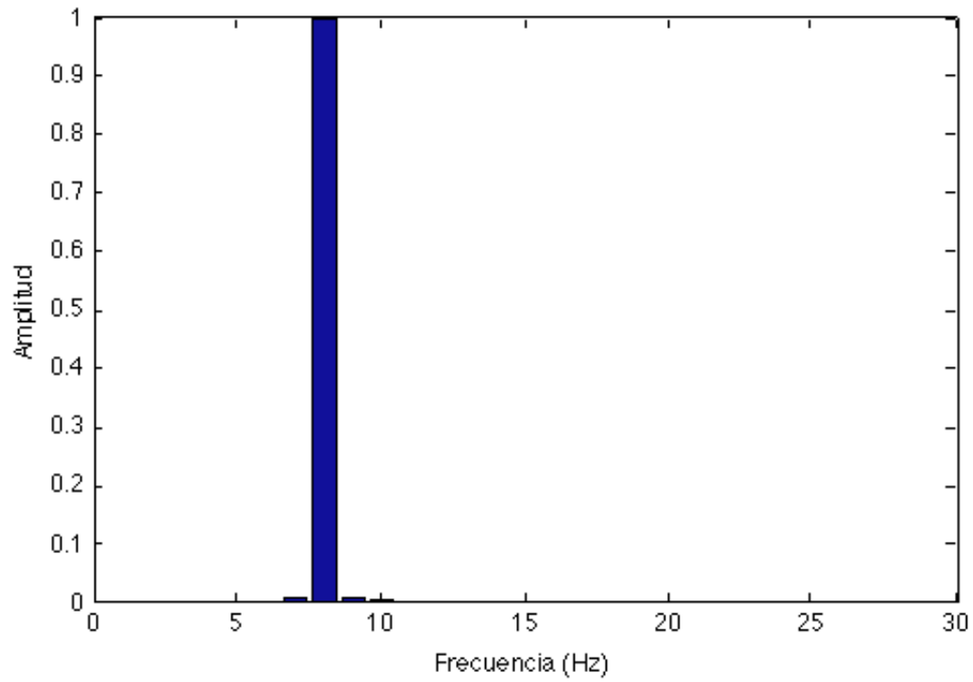
```
F = np.fft.fft(y)
N = len(F)

p= np.linspace(1,49,dtype=int, num=50)
plt.bar(x=p, height=abs(F[p])/N*2)
plt.show()
```

Ejemplo con mucho ruido

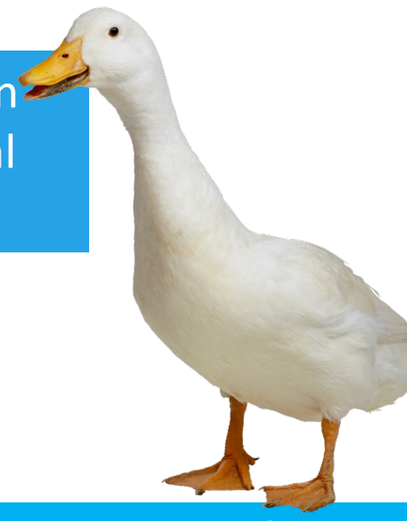
- Ejemplo de señal de 8 Hz con ruido gausiano normal





- Señal original: $\sin(2\pi \cdot 8)$

A pesar del ruido, aún es posible ver la señal que está de fondo.



Anexo

Estimación de la Elipse



- Características geométricas

Centro de Masa

Tamaño

Perímetro

Redondez

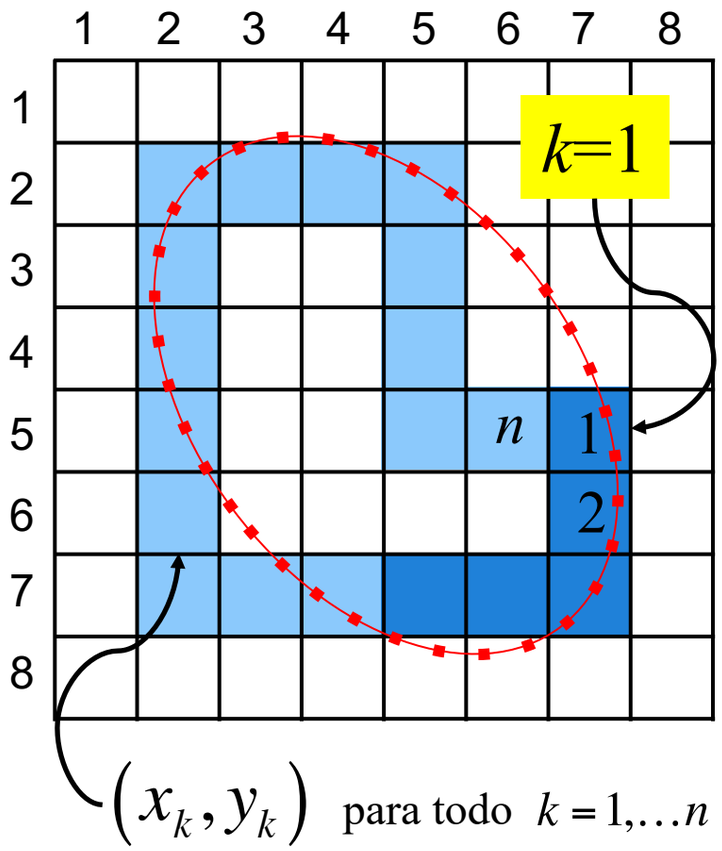
Momentos

Descriptores de Fourier

Elipse

Distancias

- Una forma para determinar la orientación de los objetos consiste en una estimación de la elipse.



Ecuación de la elipse

$$ax^2 + bxy + cy^2 + dx + ey + f = 0$$

restricción: $4ac - b^2 < 0$

Objetivo

Encontrar $\{a,b,c,d,e,f\}$ tal que:

$$\sum_{k=1}^{N}\left(ax^2 + bxy + cy^2 + dx + ey + f\right) \rightarrow \min$$

- Características geométricas

Centro de Masa

Tamaño

Perímetro

Redondez

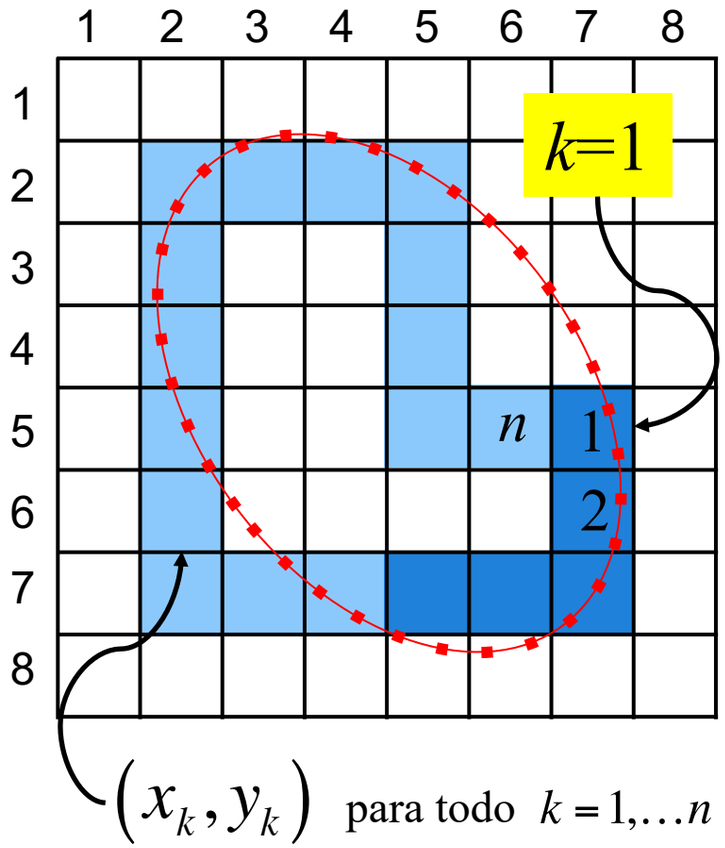
Momentos

Descriptores de Fourier

Elipse

Distancias

- Una forma para determinar la orientación de los objetos consiste en una estimación de la elipse.



Cada coordenada genera una fila en la matriz X

Método

$$\begin{bmatrix} x_1^2 & x_1y_1 & y_1^2 & x_1 & y_1 & 1 \\ x_2^2 & x_2y_2 & y_2^2 & x_2 & y_2 & 1 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ x_n^2 & x_ny_n & y_n^2 & x_n & y_n & 1 \end{bmatrix} \cdot \begin{bmatrix} a \\ b \\ c \\ d \\ e \\ f \end{bmatrix}$$

$\mathbf{X}$ $\mathbf{a}$

Objetivo

$$\|\mathbf{X} \cdot \mathbf{a}\| \rightarrow \min$$

donde $\mathbf{a} = \begin{bmatrix} a & b & c & d & e & f \end{bmatrix}^T$

- Características geométricas

Centro de Masa

Tamaño

Perímetro

Redondez

Momentos

Descriptores de Fourier

Elipse

Distancias

- Una forma para determinar la orientación de los objetos consiste en estimación de la elipse.

$$\begin{bmatrix} x_1^2 & x_1y_1 & y_1^2 & x_1 & y_1 & 1 \\ x_2^2 & x_2y_2 & y_2^2 & x_2 & y_2 & 1 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ x_n^2 & x_ny_n & y_n^2 & x_n & y_n & 1 \end{bmatrix} \cdot \begin{bmatrix} a \\ b \\ c \\ d \\ e \\ f \end{bmatrix}$$

$\mathbf{X}$ $\qquad$ $\mathbf{a}$

El algoritmo debe construir la matriz $\mathbf{X}$ a partir de las coordenadas del perímetro.

Fitzgibbon, A.W., Pilu, M and Fischer, R. B.: Direct least squares fitting of ellipses. In *Proc. of the 13th International Conference on Pattern Recognition*, pages 253–257, Vienna, September 1996

Solución matemática de Fitzgibbon (1996)

suponemos que $4ac - b^2 = 1$

$$\min_{\mathbf{a}} \|\mathbf{X} \cdot \mathbf{a}\| \quad \text{sujeto a} \quad \mathbf{a}^T\mathbf{C}\mathbf{a} = 1$$

donde $\mathbf{C} = \begin{bmatrix} 0 & 0 & 2 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 & 0 & 0 \\ 2 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$

aplicando operadores de Lagrange $\mathbf{Sa} = \lambda\mathbf{Ca}$

donde $\mathbf{S} = \mathbf{D}'\mathbf{D}$

la solución queda en el menor valor propio positivo

$[\mathbf{vec}, \mathbf{val}] = eig(\mathbf{S}, \mathbf{C})$

- Características geométricas

Centro de Masa

Tamaño

Perímetro

Redondez

Momentos

Descriptores de Fourier

Elipse

Distancias

- Una forma para determinar la orientación de los objetos consiste en estimación de la elipse.

parámetros

$$\mathbf{X}\cdot\mathbf{a} = \begin{bmatrix} x_1^2 & x_1y_1 & y_1^2 & x_1 & y_1 & 1 \\ x_2^2 & x_2y_2 & y_2^2 & x_2 & y_2 & 1 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ x_n^2 & x_ny_n & y_n^2 & x_n & y_n & 1 \end{bmatrix} \cdot \begin{bmatrix} a \\ b \\ c \\ d \\ e \\ f \end{bmatrix}$$

```
def fitzgibbon(x, y):
    n = x.shape[0]                                  # puntos de la frontera
    X = [x**2, x*y, y**2, x, y, np.ones((n,1))] # matriz X
    X = np.hstack(X)
    S = np.matmul(X.T,X)                            # matriz dispersión
    C = np.zeros((6,6))
    C[0,2]=2; C[1,1]=-1; C[2,0]=2;                  # matriz restricción

    E, V = np.linalg.eig(np.dot(np.linalg.inv(S),C))
    idx = np.argwhere(E>0 & ~np.isinf(E))
    params = V[:,idx]                               # valor propio
    return params
```

- Características geométricas

Centro de Masa

Tamaño

Perímetro

Redondez

Momentos

Descriptores de Fourier

Elipse

Distancias

- Una vez encontrados los parámetros de la elipse, debemos encontrar los parámetros de la ecuación paramétrica.

Parámetros de la Elipse

$$\mathbf{a} = \begin{bmatrix} a & b & c & d & e & f \end{bmatrix}^{\mathrm{T}}$$

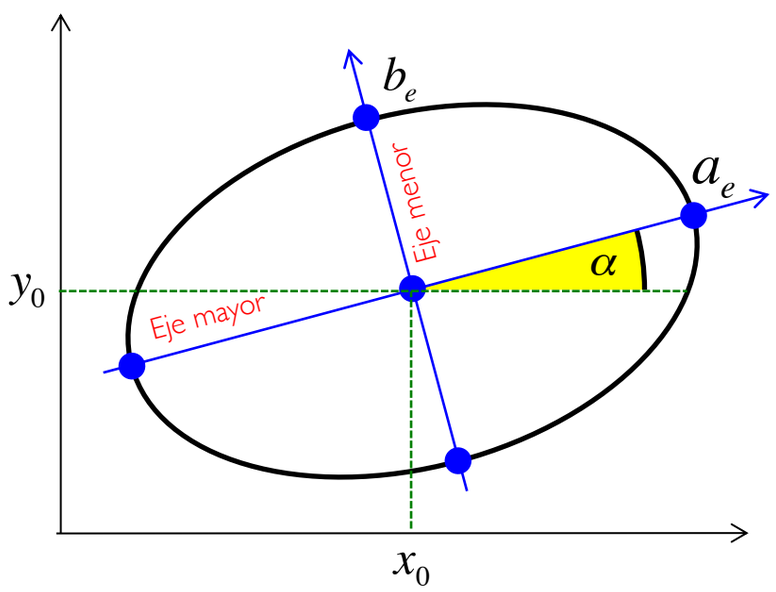
Ecuación cartesiana

$$\left(\frac{x - x_0}{a_e}\right)^2 + \left(\frac{y - y_0}{b_e}\right)^2 = 1$$

Restricción

$$4ac - b^2 = 1$$

Modelo de elipse



Pasos intermedios para la solución

$$\alpha = \frac{1}{2}\arctan\left(\frac{b}{a - c}\right)$$

$$a_p = a\cos^2(\alpha) + b\cos(\alpha)\sin(\alpha) + c\sin^2(\alpha)$$

$$b_p = a\sin^2(\alpha) - b\cos(\alpha)\sin(\alpha) + c\cos^2(\alpha)$$

$$\mathbf{T} = \begin{bmatrix} a & b/2 \\ b/2 & c \end{bmatrix} \quad t = -\frac{1}{2}\mathbf{T}^{-1}\begin{bmatrix} d \\ e \end{bmatrix} \quad v = \mathbf{t}^{\mathrm{T}} \cdot \mathbf{T} \cdot \mathbf{t}$$

$$s = \frac{1}{v - f}$$

- Características geométricas

Centro de Masa

Tamaño

Perímetro

Redondez

Momentos

Descriptores de Fourier

Elipse

Distancias

- Una vez encontrados los parámetros de la elipse, debemos encontrar los parámetros de la ecuación paramétrica.

Parámetros de la Elipse

$$\mathbf{a} = \begin{bmatrix} a & b & c & d & e & f \end{bmatrix}^{\mathrm{T}}$$

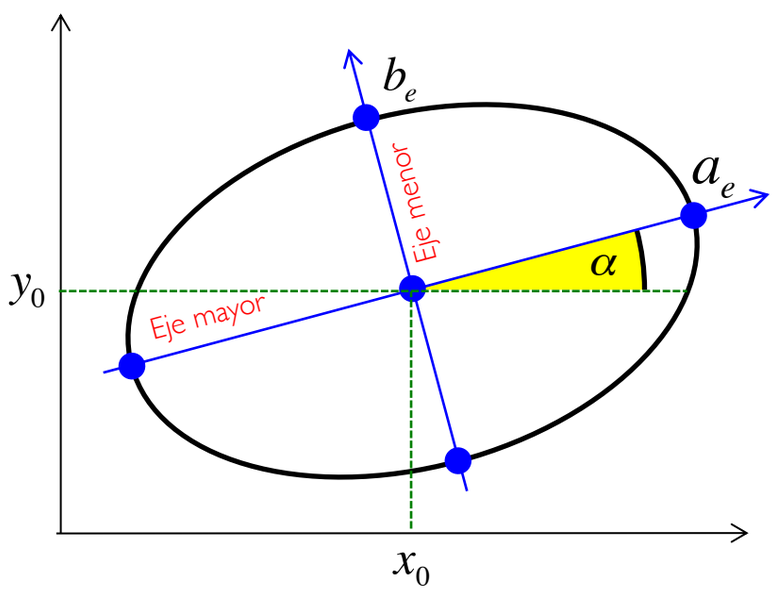
Ecuación cartesiana

$$\left(\frac{x - x_0}{a_e}\right)^2 + \left(\frac{y - y_0}{b_e}\right)^2 = 1$$

Restricción

$$4ac - b^2 = 1$$

Modelo de elipse



Solución

$$\alpha = \frac{1}{2}\arctan\left(\frac{b}{a - c}\right)$$

$$a_e = \frac{1}{\sqrt{s \cdot a_p}} \qquad b_e = \frac{1}{\sqrt{s \cdot b_p}}$$

$$x_0 = t_1 \qquad y_0 = t_2$$

$$e = \frac{\min(a_e, b_e)}{\max(a_e, b_e)}$$

Excentricidad. Si ésta es uno, la región es un círculo. En otros casos, los valores van entre cero y uno

- Características geométricas

Centro de Masa

Tamaño

Perímetro

Redondez

Momentos

Descriptores de Fourier

Elipse

Distancias

Parámetros de la Elipse

```
def parametros(x, y):

  a,b,c,d,e,f = fitzgibbon(x, y)

  alfa = np.arctan2(b, a-c)/2

  ct = np.cos(alfa);  st = np.sin(alfa)
  ap = a*ct**2 + b*ct*st + c*st**2
  cp = a*st**2 - b*ct*st + c*ct**2

  T = np.array([[a, b/2],
                [b/2, c]])
  T = T.reshape(2,2)

  t = -1* np.matmul(
          np.linalg.inv(2*T),
          np.array([d, e]).reshape(2,1))
  cx = t[0]; cy = t[1]

  val = np.matmul(t.T,np.matmul(T,t))
  scale = 1 / (val- f)

  r1 = 1/np.sqrt(scale*ap)
  r2 = 1/np.sqrt(scale*cp)

  v = np.array([r1, r2, cx, cy, alfa])
  return v
```

$$\mathbf{a} = \begin{bmatrix} a & b & c & d & e & f \end{bmatrix}^{\mathrm{T}}$$

$$\alpha = \frac{1}{2}\arctan\left(\frac{b}{a-c}\right)$$

$$a_p = a\cos^2(\alpha) + b\cos(\alpha)\sin(\alpha) + c\sin^2(\alpha)$$
$$b_p = a\sin^2(\alpha) - b\cos(\alpha)\sin(\alpha) + c\cos^2(\alpha)$$

$$\mathbf{T} = \begin{bmatrix} a & b/2 \\ b/2 & c \end{bmatrix}$$

$$t = -\frac{1}{2}\mathbf{T}^{-1}\begin{bmatrix} d \\ e \end{bmatrix}$$

$$x_0 = t_1 \quad y_0 = t_2$$

$$v = \mathbf{t}^{\mathrm{T}} \cdot \mathbf{T} \cdot \mathbf{t}$$

$$s = \frac{1}{v - f}$$

$$a_e = \frac{1}{\sqrt{s \cdot a_p}}$$

$$b_e = \frac{1}{\sqrt{s \cdot b_p}}$$

- Características geométricas

Centro de Masa

Tamaño

Perímetro

Redondez

Momentos

Descriptores de Fourier

Elipse

Distancias

- Conocidos los parámetros de la solución paramétrica, debemos emplearlos para dibujar la elipse (esto último es para plotearla)

Solución

$$\alpha = \frac{1}{2}\tan^{-1}\left(\frac{b}{a-c}\right) \qquad a_e = \frac{1}{\sqrt{s \cdot a_p}} \qquad b_e = \frac{1}{\sqrt{s \cdot b_p}}$$

$$x_0 = t_1 \qquad y_0 = t_2$$

Solución paramétrica

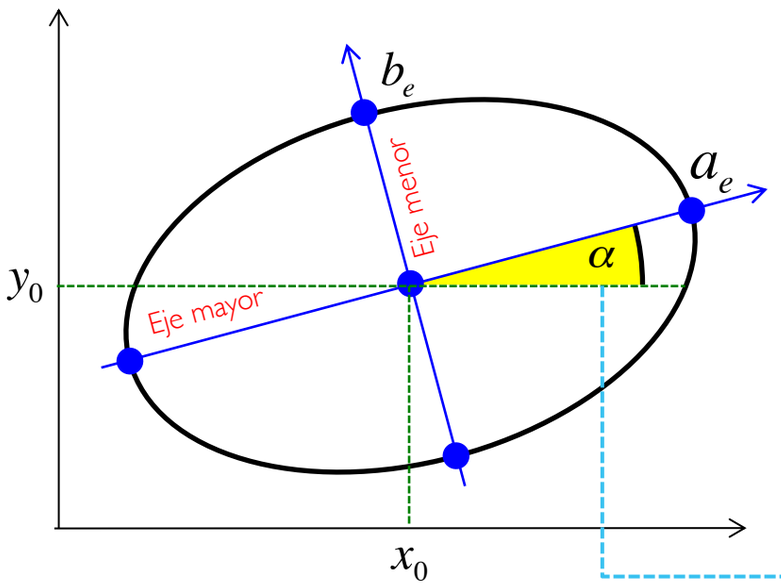
$$x = x_0 + a_e \cos(\theta)$$

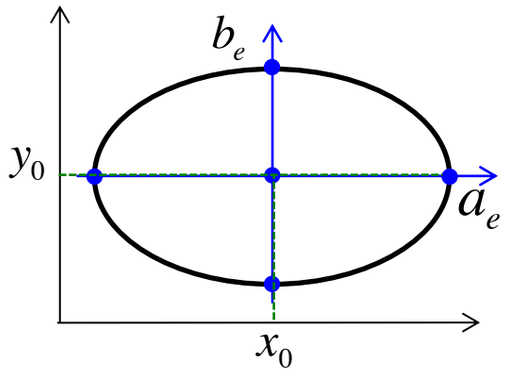
$$y = y_0 + b_e \sin(\theta)$$

La solución paramétrica utiliza cuatro parámetros $\{a_e, b_e, x_0, y_0\}$

Problema: Esta elipse está en forma horizontal → Solución: Rotar la elipse en el ángulo alfa

Modelo de elipse





$$\alpha = \frac{1}{2}\tan^{-1}\left(\frac{b}{a-c}\right)$$

- Centro de Masa
- Tamaño
- Perímetro
- Redondez
- Momentos
- Descriptores de Fourier
- **Elipse**
- Distancias

Características geométricas

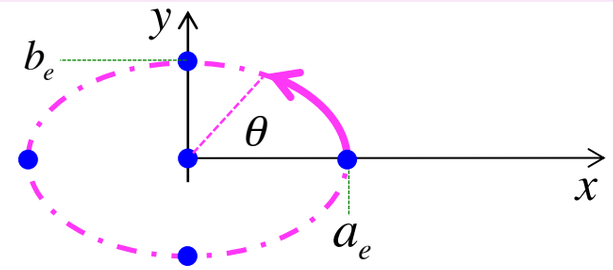
- Conocidos los parámetros de la solución paramétrica, debemos emplearlos para dibujar la elipse (esto último es para plotearla)

Pasos para rotar y trasladar

1

$$\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} a_e \cos(\theta) \\ b_e \sin(\theta) \end{bmatrix}$$

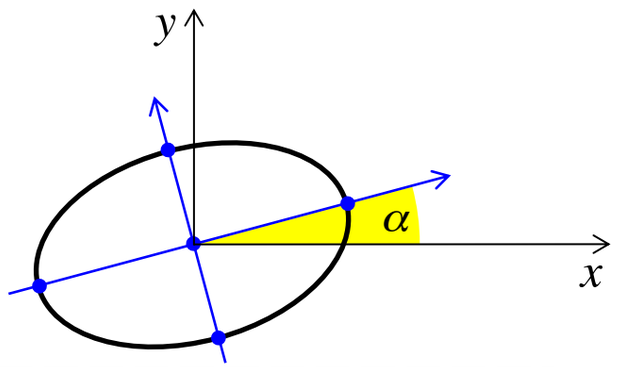
Theta es el ángulo de rotación interno. Desde cero a 360 grados



2

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos\alpha & -\sin\alpha \\ \sin\alpha & \cos\alpha \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

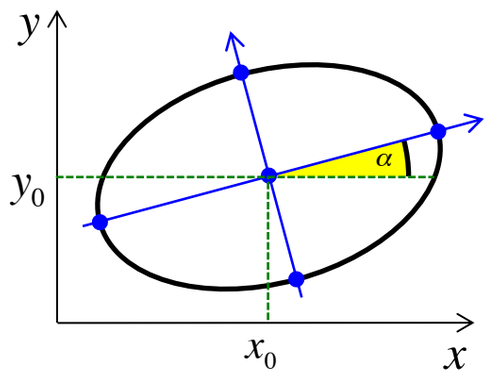
Alfa es el ángulo de giro de la elipse



3

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos\alpha & -\sin\alpha \\ \sin\alpha & \cos\alpha \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} x_0 \\ y_0 \end{bmatrix}$$

Traslación



- Características geométricas

Centro de Masa
Tamaño
Perímetro
Redondez
Momentos
Descriptores de Fourier
Elipse
Distancias

- Una forma para determinar la orientación de los objetos consiste en estimación de la elipse.

de puntos

$$v = \begin{bmatrix} a_e & b_e & x_0 & y_0 & \alpha \end{bmatrix}$$

```
def draw_ellipse(v, N=100):

    ae, be, x0, y0, alfa = v

    theta = np.linspace(0, 2*pi, N)

    R = np.array([[cos(alfa), -sin(alfa)],
                  [sin(alfa), cos(alfa)]])
    R = R.reshape(2,2)


    X = ae*np.cos(theta)
    Y = be*np.sin(theta)
    M = np.array([X,Y]).reshape(2,-1))


    CC = np.matmul(R,M)+ \
     np.matlib.repmat(np.array([v[2],v[3]]).reshape(2,1),1,N)

    return CC
```

$$\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} a_e \cos(\theta) \\ b_e \sin(\theta) \end{bmatrix}$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos\alpha & -\sin\alpha \\ \sin\alpha & \cos\alpha \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} x_0 \\ y_0 \end{bmatrix}$$